

**VÁCI SZAKKÉPZÉSI CENTRUM
BORONKAY GYÖRGY
MŰSZAKI TECHNIKUM ÉS GIMNÁZIUM**

VIZSGAREMEK

Savanyú Vivien

Nánássy Hunor Zsolt

Németh Rajmond Ádám

2026.

VÁCI SZAKKÉPZÉSI CENTRUM
BORONKAY GYÖRGY
MŰSZAKI TECHNIKUM ÉS GIMNÁZIUM



VIZSGAREMEK

Ponth

Konzulens: Wiezl Csaba

Készítette: Savanyú Vivien

Nánássy Hunor Zsolt

Németh Rajmond Ádám

Hallgatói nyilatkozat

Alulírottak, ezúton kijelentjük, hogy a vizsgaremek saját, önálló munkánk, és korábban még sehol nem került publikálásra.

Savanyú Vivien

Nánácssy Hunor
Zalán

Németh Rajmond
Ádám

Konzultációs lap

Vizsgázók neve: Savanyú Vivien, Nánássy Hunor Zsolt, Németh Rajmond Ádám

Vizsgaretek címe: Ponth

Program nyújtotta szolgáltatások:

- Vendégprofil létrehozása és kezelése (regisztráció, bejelentkezés)
- Asztalfoglalás és QR kód alapú asztal azonosítás
- Egyedi koktélok készítése és meglévők testreszabása
- Rendelés leadása és menü böngészése telefonról
- Számla kezeléssel fizetés online vagy a fő gépen keresztül
- Pincérek részére asztalok, rendelések és foglalások nyomon követése

Sorszám	A konzultáció időpontja	A konzulens aláírása
1.	2025.10.16.	
2.	2025.11.20.	
3.	2025.12.11.	
4.	2026.01.15	
5.	2026.02.19.	
6.	2026.03.26.	

A vizsgaretek beadható:

A vizsgareteket átvettem:

Vác, 2026.

Vác, 2026.

Konzulens

A szakképzést folytató
intézmény felelőse

Tartalomjegyzék

Hallgatói nyilatkozat.....	3
Konzultációs lap.....	4
1. Bevezetés	8
1.1. Témaválasztás.....	8
1.2. Követelmények.....	8
1.2.1. A vizsgarész megnevezése.....	8
1.2.2. A vizsgarész ismertetése	8
1.2.3. A szoftveralkalmazás elvárásai	9
1.2.4. A megosztott anyagnak tartalmaznia kell	9
1.2.5. A vizsgarész bemutatása	9
1.2.6. A vizsgaremek elkészítésére rendelkezésre álló idő	10
2. Fejlesztői dokumentáció.....	11
2.1. Alkalmazás ismertetése.....	11
2.2. Technológiák.....	11
2.2.1. Webes felület	11
2.2.2. Asztali alkalmazás.....	11
2.2.3. Adatbázis-kezelés	12
2.2.4. Verziókezelés.....	12
2.2.5. Fejlesztői környezet	12
2.2.6. AI	13
2.3. Adatbázis	13
2.3.1. Technológiai implementáció: MySQL és phpMyAdmin	13
2.3.2. Adatszerkezet	14
2.3.2.1. Adattáblák felsorolása.....	14
2.3.2.2. Laravel által létrehozott adattáblák felsorolása.....	14
2.3.3. Adatbázis diagram	16
2.3.4. Adattáblák.....	17
2.3.4.1. A „users” tábla.....	17
2.3.4.2. A „reservations” tábla	19
2.3.4.3. A „bills” tábla	21
2.3.4.4. Az „open_bills” tábla.....	22
2.3.4.5. A „boxes” tábla.....	23
2.3.4.6. Az „orders” tábla	24
2.3.4.7. Az „orders_extra” tábla.....	25
2.3.4.8. A „drinks” tábla.....	27

2.3.4.9.	A „quantities” tábla	29
2.3.4.10.	Az „ingredients” tábla	30
2.3.4.11.	A „costum_cocktail_ingredients” tábla	31
2.3.4.12.	A „costum_cocktail_drinks” tábla	32
2.3.4.13.	A „costum_cocktail” tábla	33
2.3.4.14.	A „cocktails” tábla	34
2.3.4.15.	A „cocktails_drinks” tábla	35
2.3.4.16.	A „cocktail_ingredients” tábla	36
2.4.	Jogosultságok, jogosultsági szintek	37
2.4.1.	Nem regisztrált felhasználó	37
2.4.2.	Regisztrált felhasználó	37
2.4.3.	Admin	38
2.5.	API dokumentáció	39
2.5.1.	OrderController	39
2.5.2.	CostumCocktailController	45
2.6.	A projekt tesztelése	48
2.6.1.	Tesztelés és minőségbiztosítás	48
2.6.1.1.	A készre jelentés kritériumai (Definition of Done):	48
2.6.1.2.	Alkalmazott minőségbiztosítási technikák	49
2.6.1.3.	Egység tesztek (Unit Tests)	49
3.	Felhasználói dokumentáció	54
3.1.	Rendszerkövetelmény	54
3.1.1.	Szoftveres feltételek	54
3.1.2.	Hardveres feltételek	54
3.1.3.	Hálózati feltételek	54
3.2.	Használati eset diagram	55
3.3.	Felhasználói felület	56
3.4.	Weboldal bemutatása	56
3.4.1.	Üdvözlő oldal	56
3.4.2.	Navigációs sáv	57
3.4.3.	Itallap – Drinks	58
3.4.4.	Bejelentkezés – Log In	59
3.4.5.	Regisztráció – Register	59
3.4.6.	Bejelentkezett felhasználó	60
3.4.6.1.	Profil	61
3.4.6.2.	Foglalás – Reservations	62
3.4.6.3.	Rendelés – Order Drink/Order more	63

3.5. Asztali alkalmazás bemutatása	65
3.5.1. Kezdőoldal	65
3.5.2. Navigációs sáv	65
3.5.3. Rendelés – Ordering	66
3.5.4. Fizetés – Paying	68
3.5.5. Foglalások – Reservations	69
4. Továbbfejlesztési lehetőségek	70
5. Irodalomjegyzék	71
5.1. Könyvek, tankönyvek.....	71
5.2. Tutoriálok, online források	72
6. Mellékletek	73
6.1. Publikus Github repository	73

1. Bevezetés

1.1. Témaválasztás

A projektünk egy kasszázási rendszert valósít meg, amely egyszerűbbé és élvezhetőbbé teszi mind a vendéglátóegységben dolgozók, mind a vendégek részére az ott eltöltött időt. A projekt alapötletét egy valós probléma inspirálta, amellyel Hunor találkozott még a nyáron. Egy vendéglátóegységben az elavult rendszer miatt sokszor rosszul került elszámolásra a borraivaló a dolgozók részére.

Végül a projektünk kissé átalakult, és nem ennek a problémának a megoldása lett a fő cél, hanem egy átfogóbb kasszázási és vendégkezelő alkalmazás megvalósítása, amely nemcsak a háttér munkát, hanem a vendégélményt is digitalizálja.

Feladatmegosztás:

- **Savanyú Vivien:** A webes felület frontendjének designolása, tesztek írása, a dokumentáció elkészítése
- **Nánássy Hunor Zalán:** Adatbázis szerkezetének megtervezése – felépítése, webes backend elkészítése és a frontend megalapozása
- **Németh Rajmond Ádám:** Adatbázis szerkezetének megtervezése – felépítése, asztali alkalmazás tervezése, fejlesztése

1.2. Követelmények

1.2.1. A vizsgarész megnevezése

Szoftverfejlesztés és -tesztelés vizsgaremek vizsgarész

1.2.2. A vizsgarész ismertetése

A vizsgázóknak minimum 2, maximum 3 fős fejlesztői csapatot alkotva kell a vizsgát megelőzően egy komplex szoftveralkalmazást lefejleszteniük.

1.2.3. A szoftveralkalmazás elvárásai

- Életszerű, valódi problémára nyújt megoldást.
- Adattárolási és -kezelési funkciókat is megvalósít.
- RESTful architektúrának megfelelő szerver és kliens oldali komponenseket egyaránt tartalmaz.
- A kliens oldali komponens vagy komponensek egyaránt alkalmasak asztali és mobil eszközökön történő használatra. Mobil eszközre kifejlesztett kliens esetén natív mobil alkalmazás, vagy azzal hozzátétőlegesen megegyező felhasználói élményt nyújtó webes kliens egyaránt alkalmazható. Asztali eszközökre fejlesztett kliens oldali komponensnél mindenképpen szükséges webes megvalósítás is, de emellett opcionálisan natív, asztali alkalmazás is a csomag része lehet. (pl. A felhasználóknak szánt interfész webes megjelenítést használ, míg az adminisztrációs felület natív asztali alkalmazásként készül el).
- A forráskódnak a tiszta kód elveinek megfelelően kell készülnie.
- A szoftver célját, komponenseinek technikai leírását, működésének műszaki feltételeit és használatának rövid bemutatását tartalmazó dokumentáció is része a csomagnak.

1.2.4. A megosztott anyagnak tartalmaznia kell

- A szoftver forráskódja.
- Natív asztali alkalmazások esetén a program telepítőkészlete.
- Az adatbázis adatbázismodell-diagramja.
- Az adatbázis export fájlja (dump).
- A szoftveralkalmazás dokumentációja.
- A tesztekhez végzett kód, valamint a teszteredmények dokumentációja.

1.2.5. A vizsgarész bemutatása

A vizsgarész során a vizsgázó gyakorlati bemutatóval összekapcsolt szóbeli előadás formájában mutatja be a:

- szoftver célját,
- műszaki megvalósítását,
- működését,

- forráskódját,
- a csapaton belüli munkamegosztást, a fejlesztési csapatban betöltött szerepét, a fejlesztés során használt projektszervezési eszközöket.

A fentieken túl maximum 3-5 perces angol nyelven tartott szóbeli előadás formájában összefoglalót ad a szoftver céljáról és működéséről, valamint angolul válaszol a vizsgáztató végfelhasználói szerepben feltett maximum 2-3 kérdésére.

Amennyiben a munkacsoport más tagjai is azonos csoportban vizsgáznak, akkor a bemutatót közösen is megtarthatják, de ebben az esetben is biztosítani kell, hogy minden vizsgázó egyenlő arányban vegyen részt a bemutatóban, illetve minden vizsgázónak önállóan kell bemutatnia a saját feladat részét magyarul és angolul egyaránt.

1.2.6. A vizsgaremek elkészítésére rendelkezésre álló idő

A vizsgaremeket a záróvizsga tanévében kell a vizsgázóknak elkészítenie.

2. Fejlesztői dokumentáció

2.1. Alkalmazás ismertetése

A Ponth egy komplex vendéglátóipari rendszer, amely egy C# alapú asztali adminisztrációs alkalmazásból és egy a vendégek részére QR-kóddal elérhető online rendelési felületből áll. A Ponth célja a koktélbárok munkafolyamatainak digitalizálása. Az alkalmazás lehetővé teszi a dolgozók számára a rendelések és asztalfoglalások nyilvántartását és kezelését, valamint a vendégek számára az asztalfoglalást, a rendelések leadását és a számlájuk nyomon követését.

2.2. Technológiák

2.2.1. Webes felület

Laravel PHP: Egy nyílt forráskódú, PHP-alapú webes keretrendszer, amely az MVC (Model-View-Controller) architektúrát követve elegáns szintaxist és kész megoldásokat kínál a modern webalkalmazások backend fejlesztéséhez.

React (Vite + JSX): Egy deklaratív JavaScript-könyvtár felhasználói felületek építéséhez, amely JSX szintaxist használ a komponensalapú fejlesztéshez, a Vite eszköz segítségével pedig villámgyors fejlesztői környezetet és optimalizált összeállítást biztosít.¹

2.2.2. Asztali alkalmazás

C#: A Microsoft által kifejlesztett, modern, objektumorientált programozási nyelv, amely a .NET keretrendszeren futva lehetővé teszi asztali, mobil- és webes alkalmazások (például ASP.NET) hatékony fejlesztését.

SQL: A strukturált lekérdező nyelv (Structured Query Language), amely a relációs adatbázis-kezelő rendszerekben tárolt adatok lekérdezésére, kezelésére és módosítására szolgáló szabványosított nyelv.

¹[React](#)

2.2.3. Adatbázis-kezelés

MySQL: A világ legnépszerűbb nyílt forráskódú relációs adatbázis-kezelő rendszere (RDBMS), amely SQL nyelvet használ az adatok hatékony tárolására, rendszerezésére és kiszolgálására webes környezetben.

phpMyAdmin: Egy ingyenes és nyílt forráskódú, PHP nyelven írt webes eszköz, amely grafikus felületet biztosít a MySQL és MariaDB adatbázisok kezeléséhez, lehetővé téve a táblák létrehozását, lekérdezések futtatását és az adatok módosítását közvetlenül a böngészőből.

2.2.4. Verziókezelés

Git: Elosztott verziókezelő rendszer, amely nyomon követi a forráskódban történt változtatásokat, lehetővé teszi a korábbi verziókhoz való visszatérést és a párhuzamos munkavégzést.²

GitHub: Felhőalapú platform a Git tárolók hosztolására, amely közösségi fejlesztési és automatizációs funkciókkal egészíti ki a verziókezelést, biztosítva a kód biztonságos tárolását és elérhetőségét.

2.2.5. Fejlesztői környezet

Visual Studio Code: A Microsoft által fejlesztett, ingyenes forráskód-szerkesztő, amely bővítményei révén integrált környezetet biztosított a teljes munkafolyamathoz, támogatva a front-end és back-end kódolást, valamint a verziókezelést.

Visual Studio 2022: A Microsoft legmodernebb, 64 bites integrált fejlesztői környezete (IDE), amely kiterjedt eszközkészletet biztosít a .NET alapú alkalmazások fejlesztéséhez, teszteléséhez és közzétételéhez, olyan intelligens funkciókkal segítve a munkát, mint az AI-alapú kódkiegészítés (IntelliCode) és a fejlett hibakereső eszközök.

² [Korom Richárd oktatás](#)

2.2.6.AI

Gemini: A Google legfejlettebb multimodális mesterséges intelligencia modellje, amely képes szöveges tartalom, programkód és vizuális információk elemzésére, segítve a fejlesztési folyamatokat az automatizált kódgenerálástól a komplex algoritmusok optimalizálásáig.

Claude Opus: Az Anthropic legerősebb nagy nyelvi modellje, amely kiemelkedő logikai képességekkel és árnyalt szövegértéssel rendelkezik, lehetővé téve a nagy mennyiségű forráskód precíz elemzését, valamint a technikai dokumentációk és rendszerszintű architektúrák kidolgozását.

2.3. Adatbázis

2.3.1. Technológiai implementáció: MySQL és phpMyAdmin

A fejlesztés során a világ legnépszerűbb nyílt forráskódú relációs adatbázis-kezelő rendszerét, a MySQL-t alkalmaztuk. Az adatbázis adminisztrációjához és a sémák vizuális kezeléséhez a phpMyAdmin eszközt használtuk, amely az alábbi módszertani előnyöket biztosította:

- **Hatékony sémakezelés:** A phpMyAdmin grafikus felületén keresztül precízen definiálhatóak voltak a táblák struktúrái, az adattípusok és az indexelési szabályok, gyorsítva a fejlesztési ciklust.
- **Relációs integritás:** A MySQL motor (InnoDB) segítségével kényszerítettük ki a táblák közötti kapcsolatokat, garantálva, hogy ne maradhassanak „árva” rekordok például egy ital törlése esetén a koktéltreceptekben.
- **Lekérdezés-optimalizálás:** A beépített SQL-konzol lehetővé tette a komplex lekérdezések (JOIN-ok, aggregálások) közvetlen tesztelését és finomhangolását, mielőtt azok az alkalmazás forráskódjába kerültek volna.
- **Adatbiztonság és exportálás:** Az eszköz rugalmas mentési (dump) lehetőségeket biztosított, így az adatbázis állapota bármikor archiválható és visszaállítható volt a fejlesztési fázisok során.

A MySQL stabilitása és a phpMyAdmin által nyújtott átlátható kezelőfelület együttesen olyan robusztus adatbázis-réteget eredményezett, amely képes kiszolgálni a Ponth valós idejű igényeit, legyen szó készletellenőrzésről vagy asztalfoglalásról.

2.3.2. Adatszerkezet

- Adatbázis neve: ponth
- Tárolómotor: InnoDB
- Illesztés: utf8_hungarian_ci

2.3.2.1. Adattáblák felsorolása

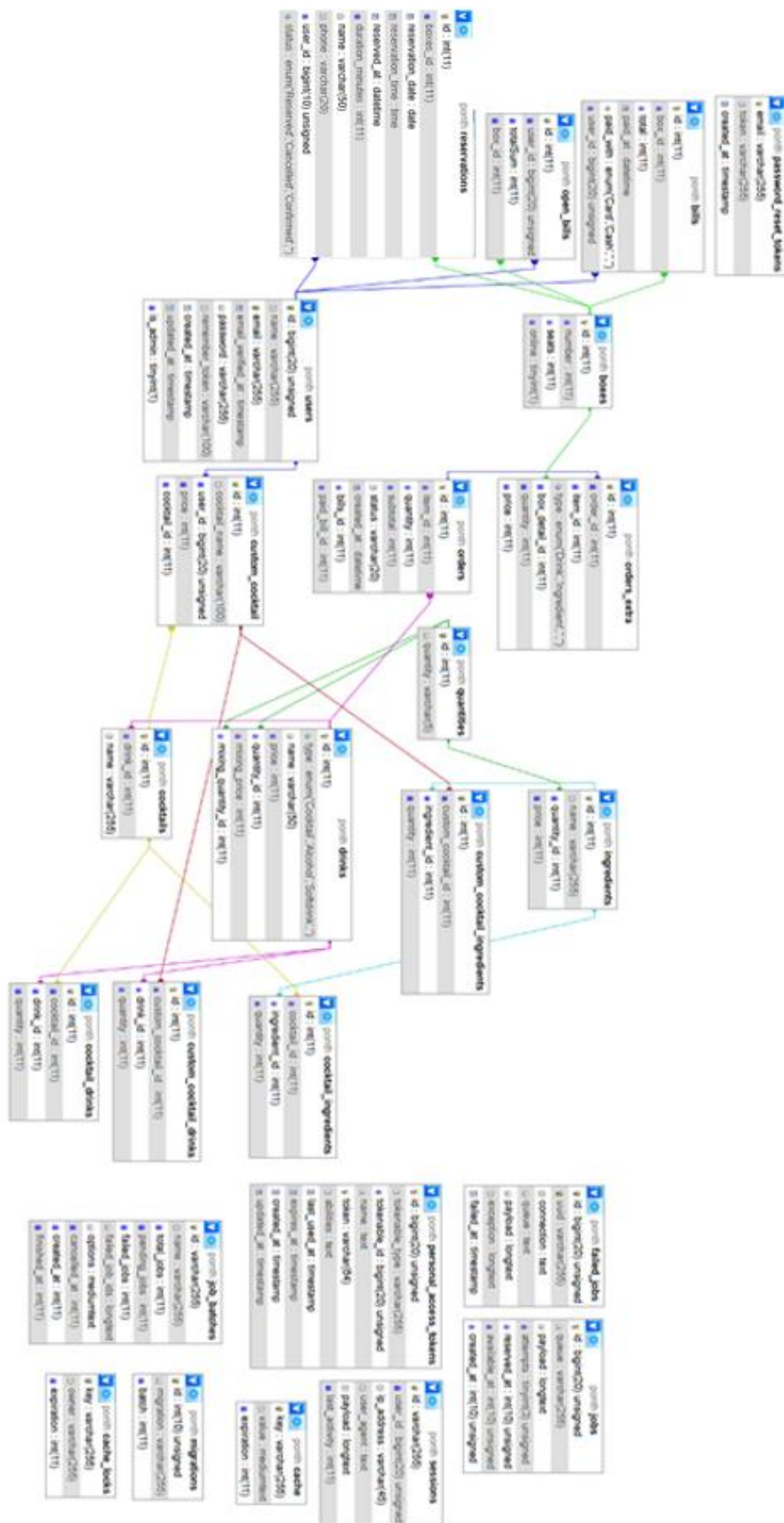
- users
- reservations
- bills
- open_bills
- boxes
- orders
- orders_extra
- drinks
- quantities
- ingredients
- custom_cocktail_ingredients
- custom_cocktail_drinks
- custom_cocktail
- cocktails
- cocktail_drinks
- cocktail_ingredients

2.3.2.2. Laravel által létrehozott adattáblák felsorolása

- cache_locks
- sessions
- cache
- migrations
- failed_jobs

- jobs
- job_batches
- personal_access_tokens
- password_reset_tokens

2.3.3. Adatbázis diagram



2.3.4. Adattáblák

2.3.4.1. A „users” tábla

Ez a tábla tartalmazza a felhasználókat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra	Művelet
☐	1 id 🔑	bigint(20)		UNSIGNED	Nem	Nincs		AUTO_INCREMENT	Módosítás Eldobás Több
☐	2 name	varchar(255)	utf8mb4_unicode_ci		Nem	Nincs			Módosítás Eldobás Több
☐	3 email 📧	varchar(255)	utf8mb4_unicode_ci		Nem	Nincs			Módosítás Eldobás Több
☐	4 email_verified_at	timestamp			Igen	NULL			Módosítás Eldobás Több
☐	5 password	varchar(255)	utf8mb4_unicode_ci		Nem	Nincs			Módosítás Eldobás Több
☐	6 remember_token	varchar(100)	utf8mb4_unicode_ci		Igen	NULL			Módosítás Eldobás Több
☐	7 created_at	timestamp			Igen	NULL			Módosítás Eldobás Több
☐	8 updated_at	timestamp			Igen	NULL			Módosítás Eldobás Több
☐	9 is_admin	tinyint(1)			Nem	0			Módosítás Eldobás Több

- *id*: a felhasználók azonosítója, tábla elsődleges kulcsa
- *name*: a felhasználók neve
- *email*: a felhasználók e-mail címe
- *email_verified_at*: az az idő, amikor a felhasználó megerősítette az e-mail címét
- *password*: a felhasználó által beállított jelszó
- *remember_token*:
- *created_at*: a fiók létrehozásának ideje
- *updated_at*:
- *is_admin*: van-e admin jogosultsága a felhasználónak

Tábla létrehozása:

```
CREATE TABLE `users` (  
  `id` bigint(20) UNSIGNED NOT NULL,  
  `name` varchar(255) NOT NULL,  
  `email` varchar(255) NOT NULL,  
  `email_verified_at` timestamp NULL DEFAULT NULL,  
  `password` varchar(255) NOT NULL,  
  `remember_token` varchar(100) DEFAULT NULL,  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  `is_admin` tinyint(1) NOT NULL DEFAULT 0  
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_unicode_ci;  
ALTER TABLE `users`
```

```
ADD PRIMARY KEY (`id`),  
ADD UNIQUE KEY `users_email_unique` (`email`);  
ALTER TABLE `users`  
MODIFY `id` bigint(20) UNSIGNED NOT NULL AUTO_INCREMENT;  
COMMIT;
```

2.3.4.2. A „reservations” tábla

Ez a tábla tartalmazza az asztalfoglalásokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
2	boxes_id	int(11)			Igen	NULL		
3	reservation_date	date			Igen	NULL		
4	reservation_time	time			Igen	NULL		
5	reserved_at	datetime			Nem	Nincs		
6	duration_minutes	int(11)			Nem	Nincs		
7	name	varchar(50)	utf8mb4_general_ci		Igen	NULL		
8	phone	varchar(20)	utf8mb4_general_ci		Igen	NULL		
9	user_id	bigint(10)		UNSIGNED	Igen	NULL		
10	status	enum('Reserved', 'Cancelled', 'Confirmed', '')	utf8mb4_general_ci		Nem	Nincs	pl.: -foglalva, -lemondva, -megerositve	

- *id*: a foglalás azonosítója, a tábla elsődleges kulcsa
- *boxes_id*: a foglalt asztal azonosítója
- *reservation_date*: a foglalás napja
- *reservation_time*: a foglalás időpontja
- *reserved_at*: a foglalás rögzítésének időpontja
- *duration_minutes*: a foglalás hossza percekben
- *name*: a foglaló személy neve
- *phone*: a foglaló telefonszáma
- *user_id*: a foglaláshoz tartozó regisztrált felhasználó azonosítója
- *status*: a foglalás állapota (pl. foglalva, lemondva, megerosítve)

Tábla létrehozása:

```
CREATE TABLE `reservations` (  
  `id` int(11) NOT NULL,  
  `boxes_id` int(11) DEFAULT NULL,  
  `reservation_date` date DEFAULT NULL,  
  `reservation_time` time DEFAULT NULL,  
  `reserved_at` datetime NOT NULL,  
  `duration_minutes` int(11) NOT NULL,  
  `name` varchar(50) DEFAULT NULL,  
  `phone` varchar(20) DEFAULT NULL,  
  `user_id` bigint(10) UNSIGNED DEFAULT NULL,  
  `status` enum('Reserved','Cancelled','Confirmed','') NOT NULL COMMENT  
  'pl.: -foglalva,-lemondva,-megerositve'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;
```

```
ALTER TABLE `reservations`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `boxes_id` (`boxes_id`),  
  ADD KEY `user_id` (`user_id`);  
ALTER TABLE `reservations`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `reservations`  
  ADD CONSTRAINT `reservations_ibfk_1` FOREIGN KEY (`boxes_id`)  
REFERENCES `boxes` (`id`),  
  ADD CONSTRAINT `reservations_ibfk_2` FOREIGN KEY (`user_id`)  
REFERENCES `users` (`id`);  
COMMIT;
```

2.3.4.3. A „bills” tábla

Ez a tábla tartalmazza a már kifizetett számlák adatait.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
2	box_id 🔑	int(11)			Nem	Nincs		
3	total	int(11)			Nem	Nincs		
4	paid_at	datetime			Nem	Nincs		
5	paid_with	enum('Card', 'Cash', '', '')	utf8mb4_general_ci		Nem	Nincs		
6	user_id 🔑	bigint(20)		UNSIGNED	Igen	NULL		

- *id*: a számla azonosítója, a tábla elsődleges kulcsa
- *box_id*: az érintett asztal azonosítója
- *total*: a számla végösszege
- *paid_at*: a fizetés időpontja
- *paid_with*: a fizetés módja (Card, Cash)
- *user_id*: a számlához tartozó felhasználó azonosítója

Tábla létrehozása:

```
CREATE TABLE `bills` (  
  `id` int(11) NOT NULL,  
  `box_id` int(11) NOT NULL,  
  `total` int(11) NOT NULL,  
  `paid_at` datetime NOT NULL,  
  `paid_with` enum('Card','Cash','','') NOT NULL  
  `user_id` bigint(20) UNSIGNED DEFAULT NULL,  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `bills`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `box_id` (`box_id`),  
  ADD KEY `user_id` (`user_id`);  
ALTER TABLE `bills`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `bills`  
  ADD CONSTRAINT `bills_ibfk_1` FOREIGN KEY (`box_id`) REFERENCES  
  `boxes` (`id`),  
  ADD CONSTRAINT `bills_ibfk_2` FOREIGN KEY (`user_id`) REFERENCES  
  `users` (`id`); COMMIT;
```

2.3.4.4. Az „open_bills” tábla

Ez a tábla a jelenleg aktív, még le nem zárt számlákat tartalmazza.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
2	user_id 🔑	bigint(20)	UNSIGNED		Igen	NULL		
3	totalSum	int(11)			Nem	Nincs		
4	box_id 🔑	int(11)			Nem	Nincs		

- *id*: a nyitott számla azonosítója, a tábla elsődleges kulcsa
- *user_id*: a számlához tartozó felhasználó azonosítója
- *totalSum*: az eddigi fogyasztás összege
- *box_id*: az asztal azonosítója

Tábla létrehozása:

```
CREATE TABLE `open_bills` (  
  `id` int(11) NOT NULL,  
  `user_id` bigint(20) UNSIGNED DEFAULT NULL,  
  `totalSum` int(11) NOT NULL,  
  `box_id` int(11) NOT NULL,  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `open_bills`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `user_id` (`user_id`),  
  ADD KEY `box_id` (`box_id`);  
ALTER TABLE `open_bills`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `open_bills`  
  ADD CONSTRAINT `open_bills_ibfk_1` FOREIGN KEY (`user_id`)  
REFERENCES `users` (`id`),  
  ADD CONSTRAINT `open_bills_ibfk_2` FOREIGN KEY (`box_id`)  
REFERENCES `boxes` (`id`);  
COMMIT;
```

2.3.4.5. A „boxes” tábla

Ez a tábla tartalmazza az asztalokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id 	int(11)			Nem	<i>Nincs</i>		AUTO_INCREMENT
☐	2 number	int(11)			Igen	<i>NULL</i>		
☐	3 seats	int(11)			Igen	<i>NULL</i>		
☐	4 online	tinyint(1)			Nem	<i>Nincs</i>		

- *id*: az asztal azonosítója, a tábla elsődleges kulcsa
- *number*: az asztal sorszáma
- *seats*: az asztalhoz tartozó férőhelyek száma
- *online*: az asztal foglalható-e online

Tábla létrehozása:

```
CREATE TABLE `boxes` (  
  `id` int(11) NOT NULL,  
  `number` int(11) DEFAULT NULL,  
  `seats` int(11) DEFAULT NULL,  
  `online` tinyint(1) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `boxes`  
  ADD PRIMARY KEY (`id`);  
ALTER TABLE `boxes`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
COMMIT;
```

2.3.4.6. Az „orders” tábla

Ez a tábla tartalmazza a leadott rendeléseket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id	int(11)			Nem	Nincs		AUTO_INCREMENT
☐	2 box_id	int(11)			Igen	NULL		
☐	3 item_id	int(11)			Igen	NULL		
☐	4 quantity	int(11)			Igen	NULL		
☐	5 subtotal	int(11)			Igen	NULL		
☐	6 status	varchar(20)	utf8mb4_general_ci		Nem	new		

- *id*: a rendelési tétel azonosítója, a tábla elsődleges kulcsa
- *box_id*: az asztal azonosítója, ahonnan a rendelés érkezett
- *item_id*: a rendelt ital azonosítója
- *quantity*: a rendelt mennyiség
- *subtotal*: a tétel részösszege
- *status*: a tétel állapota (pl. new, served)

Tábla létrehozása:

```
CREATE TABLE `orders` (  
  `id` int(11) NOT NULL,  
  `box_id` int(11) DEFAULT NULL,  
  `item_id` int(11) DEFAULT NULL,  
  `quantity` int(11) DEFAULT NULL,  
  `subtotal` int(11) DEFAULT NULL,  
  `status` varchar(20) NOT NULL DEFAULT 'new'  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `orders`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `item_id` (`item_id`);  
ALTER TABLE `orders`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `orders`  
  ADD CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`item_id`) REFERENCES  
  `drinks` (`id`);  
COMMIT;
```

2.3.4.7. Az „orders_extra” tábla

Ez a tábla tartalmazza a rendelésekhez tartozó extra összetevőket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	order_id 🔑	int(11)			Nem	Nincs		
☐ 3	item_id 🔑	int(11)			Nem	Nincs		
☐ 4	type	enum('Drink', 'Ingredient', '', '')	utf8mb4_general_ci		Nem	Nincs		
☐ 5	box_detail_id 🔑	int(11)			Nem	Nincs		
☐ 6	quantity	int(11)			Nem	Nincs		
☐ 7	price	int(11)			Nem	Nincs		

- *id*: az extra tétel azonosítója, a tábla elsődleges kulcsa
- *order_id*: a fő rendelési tétel azonosítója
- *item_id*: a hozzáadott tétel (ital vagy összetevő) azonosítója
- *type*: az extra típusa (Drink, Ingredient)
- *box_detail_id*: az asztal azonosítója
- *quantity*: mennyiség
- *price*: ár

Tábla létrehozása:

```
CREATE TABLE `orders_extra` (  
  `id` int(11) NOT NULL,  
  `order_id` int(11) NOT NULL,  
  `item_id` int(11) NOT NULL,  
  `type` enum('Drink','Ingredient','','') NOT NULL,  
  `box_detail_id` int(11) NOT NULL,  
  `quantity` int(11) NOT NULL,  
  `price` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `orders_extra`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `item_id` (`item_id`),  
  ADD KEY `box_id` (`box_detail_id`),  
  ADD KEY `item_id_2` (`item_id`),  
  ADD KEY `order_id` (`order_id`),  
  ADD KEY `item_id_3` (`item_id`),  
  ADD KEY `item_id_4` (`item_id`);
```

```
ALTER TABLE `orders_extra`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `orders_extra`  
  ADD CONSTRAINT `orders_extra_ibfk_4` FOREIGN KEY (`box_detail_id`)  
REFERENCES `boxes` (`id`),  
  ADD CONSTRAINT `orders_extra_ibfk_5` FOREIGN KEY (`order_id`)  
REFERENCES `orders` (`id`);  
COMMIT;
```

2.3.4.8. A „drinks” tábla

Ez a tábla tartalmazza az italokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	type	enum('Cocktail','Alcohol','Softdrink',")	utf8mb4_general_ci		Nem	Nincs		
☐ 3	name	varchar(50)	utf8mb4_general_ci		Nem	Nincs		
☐ 4	price	int(11)			Nem	Nincs		
☐ 5	quantity_id	int(11)			Nem	Nincs		
☐ 6	mixing_price	int(11)			Nem	Nincs		
☐ 7	mixing_quantity_id	int(11)			Nem	Nincs		

- *id*: az ital azonosítója, a tábla elsődleges kulcsa
- *type*: az ital típusa
- *name*: az ital megnevezése
- *price*: az ital alapára
- *quantity_id*: az ital mennyiségi egységének azonosítója
- *mixing_price*: keverési felár
- *mixing_quantity_id*: keverési mennyiségi egység azonosítója

Tábla létrehozása:

```
CREATE TABLE `drinks` (  
  `id` int(11) NOT NULL,  
  `type` enum('Cocktail','Alcohol','Softdrink','') NOT NULL,  
  `name` varchar(50) NOT NULL,  
  `price` int(11) NOT NULL,  
  `quantity_id` int(11) NOT NULL,  
  `mixing_price` int(11) NOT NULL,  
  `mixing_quantity_id` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `drinks`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `mixing_quantity_id` (`mixing_quantity_id`),  
  ADD KEY `quantity_id` (`quantity_id`);  
ALTER TABLE `drinks`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `drinks`  
  ADD CONSTRAINT `drinks_ibfk_1` FOREIGN KEY (`quantity_id`)  
REFERENCES `quantities` (`id`),
```

```
ADD CONSTRAINT `drinks_ibfk_2` FOREIGN KEY (`mixing_quantity_id`)
REFERENCES `quantities` (`id`);
COMMIT;
```

2.3.4.9. A „quantities” tábla

Ez a tábla tartalmazza a mértékegységeket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id 	int(11)			Nem	Nincs		AUTO_INCREMENT
☐	2 quantity	varchar(5)	utf8mb4_general_ci		Nem	Nincs		

- *id*: a mértékegység azonosítója, a tábla elsődleges kulcsa
- *quantity*: a mértékegység neve (dl, ml, piece stb.)

Tábla létrehozása:

```
CREATE TABLE `quantities` (  
  `id` int(11) NOT NULL,  
  `quantity` varchar(5) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `quantities`  
  ADD PRIMARY KEY (`id`);  
ALTER TABLE `quantities`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
COMMIT;
```

2.3.4.10. Az „ingredients” tábla

Ez a tábla tartalmazza az összetevőket.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id 🔑	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	name	varchar(255)	utf8mb4_general_ci		Igen	NULL		
☐ 3	quantity_id 🔗	int(11)			Nem	Nincs		
☐ 4	price	int(11)			Igen	NULL		

- *id*: az összetevő azonosítója, a tábla elsődleges kulcsa
- *name*: az összetevő neve
- *quantity_id*: mértékegység azonosítója
- *price*: az összetevő ára

Tábla létrehozása:

```
CREATE TABLE `ingredients` (  
  `id` int(11) NOT NULL,  
  `name` varchar(255) DEFAULT NULL,  
  `quantity_id` int(11) NOT NULL,  
  `price` int(11) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `ingredients`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `quantity_id` (`quantity_id`);  
ALTER TABLE `ingredients`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `ingredients`  
  ADD CONSTRAINT `ingredients_ibfk_1` FOREIGN KEY (`quantity_id`)  
REFERENCES `quantities` (`id`);  
COMMIT;
```

2.3.4.11. A „costum_cocktail_ingredients” tábla

Ez a tábla tartalmazza az egyedi koktélok extra összetevőit.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	custom_cocktail_id 	int(11)			Nem	Nincs		
☐ 3	ingredient_id 	int(11)			Nem	Nincs		
☐ 4	quantity	int(11)			Nem	Nincs		

- *id*: azonosító, elsődleges kulcs
- *custom_cocktail_id*: a kapcsolódó egyedi koktél azonosítója
- *ingredient_id*: az összetevő azonosítója
- *quantity*: a mennyiség

Tábla létrehozása:

```
CREATE TABLE `custom_cocktail_ingredients` (  
  `id` int(11) NOT NULL,  
  `custom_cocktail_id` int(11) NOT NULL,  
  `ingredient_id` int(11) NOT NULL,  
  `quantity` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `custom_cocktail_ingredients`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `costume_cocktail_id` (`custom_cocktail_id`),  
  ADD KEY `ingredient_id` (`ingredient_id`);  
ALTER TABLE `custom_cocktail_ingredients`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `custom_cocktail_ingredients`  
  ADD CONSTRAINT `custom_cocktail_ingredients_ibfk_1` FOREIGN KEY  
  (`custom_cocktail_id`) REFERENCES `custom_cocktail` (`id`),  
  ADD CONSTRAINT `custom_cocktail_ingredients_ibfk_2` FOREIGN KEY  
  (`ingredient_id`) REFERENCES `ingredients` (`id`);  
COMMIT;
```

2.3.4.12. A „costum_cocktail_drinks” tábla

Ez a tábla tartalmazza az egyedi koktélokba kerülő italokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id 	int(11)			Nem	Nincs		AUTO_INCREMENT
☐	2 custom_cocktail_id 	int(11)			Nem	Nincs		
☐	3 drink_id 	int(11)			Nem	Nincs		

- *id*: azonosító, elsődleges kulcs
- *custom_cocktail_id*: az egyedi koktél azonosítója
- *drink_id*: a felhasznált ital azonosítója

Tábla létrehozása:

```
CREATE TABLE `custom_cocktail_drinks` (  
  `id` int(11) NOT NULL,  
  `custom_cocktail_id` int(11) NOT NULL,  
  `drink_id` int(11) NOT NULL,  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `custom_cocktail_drinks`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `costume_cocktail_id` (`custom_cocktail_id`),  
  ADD KEY `drink_id` (`drink_id`);  
ALTER TABLE `custom_cocktail_drinks`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `custom_cocktail_drinks`  
  ADD CONSTRAINT `custom_cocktail_drinks_ibfk_1` FOREIGN KEY  
  (`custom_cocktail_id`) REFERENCES `custom_cocktail` (`id`),  
  ADD CONSTRAINT `custom_cocktail_drinks_ibfk_2` FOREIGN KEY  
  (`drink_id`) REFERENCES `drinks` (`id`);  
COMMIT;
```

2.3.4.13. A „costum_cocktail” tábla

Ez a tábla tartalmazza az egyedi koktélokat.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	cocktail_name	varchar(100)	utf8mb4_general_ci		Nem	Nincs		
☐ 3	user_id	bigint(20)		UNSIGNED	Nem	Nincs		
☐ 4	price	int(11)			Nem	Nincs		
☐ 5	cocktail_id	int(11)			Nem	Nincs		

- *id*: az egyedi koktél azonosítója, a tábla elsődleges kulcsa
- *cocktail_name*: az egyedi koktél megnevezése
- *user_id*: a koktélt létrehozó felhasználó azonosítója
- *price*: az egyedi koktél ára
- *cocktail_id*: az alapul szolgáló koktél azonosítója

Tábla létrehozása:

```
CREATE TABLE `custom_cocktail` (  
  `id` int(11) NOT NULL,  
  `cocktail_name` varchar(100) NOT NULL,  
  `user_id` bigint(20) UNSIGNED NOT NULL,  
  `price` int(11) NOT NULL,  
  `cocktail_id` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `custom_cocktail`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `user_id` (`user_id`),  
  ADD KEY `cocktail_id` (`cocktail_id`);  
ALTER TABLE `custom_cocktail`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `custom_cocktail`  
  ADD CONSTRAINT `custom_cocktail_ibfk_2` FOREIGN KEY (`user_id`)  
REFERENCES `users` (`id`),  
  ADD CONSTRAINT `custom_cocktail_ibfk_3` FOREIGN KEY (`cocktail_id`)  
REFERENCES `cocktails` (`id`);  
COMMIT;
```

2.3.4.14. A „cocktails” tábla

Ez a tábla tartalmazza a koktélokot.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id	int(11)			Nem	Nincs		AUTO_INCREMENT
☐	2 drink_id	int(11)			Nem	Nincs		
☐	3 name	varchar(255)	utf8mb4_general_ci		Igen	NULL		

- *id*: a koktél azonosítója, elsődleges kulcs
- *drink_id*: a benne lévő ital azonosítója
- *name*: a koktél neve

Tábla létrehozása:

```
CREATE TABLE `cocktails` (  
  `id` int(11) NOT NULL,  
  `drink_id` int(11) NOT NULL,  
  `name` varchar(255) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `cocktails`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `drinks_id` (`drink_id`),  
  ADD KEY `drink_id` (`drink_id`);  
ALTER TABLE `cocktails`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `cocktails`  
  ADD CONSTRAINT `cocktails_ibfk_1` FOREIGN KEY (`drink_id`) REFERENCES  
  `drinks` (`id`);  
COMMIT;
```

2.3.4.15. A „cocktails_drinks” tábla

Ez a tábla tartalmazza a koktélokban lévő italt.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐ 1	id 	int(11)			Nem	Nincs		AUTO_INCREMENT
☐ 2	cocktail_id 	int(11)			Nem	Nincs		
☐ 3	drink_id 	int(11)			Nem	Nincs		
☐ 4	quantity	int(11)			Nem	Nincs		

- *id*: azonosító, elsődleges kulcs
- *cocktail_id*: a koktél azonosítója
- *drink_id*: az ital azonosítója
- *quantity*: mennyiség

Tábla létrehozása:

```
CREATE TABLE `cocktail_drinks` (  
  `id` int(11) NOT NULL,  
  `cocktail_id` int(11) NOT NULL,  
  `drink_id` int(11) NOT NULL,  
  `quantity` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `cocktail_drinks`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `cocktail_id` (`cocktail_id`),  
  ADD KEY `drink_id` (`drink_id`);  
ALTER TABLE `cocktail_drinks`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `cocktail_drinks`  
  ADD CONSTRAINT `cocktail_drinks_ibfk_3` FOREIGN KEY (`cocktail_id`)  
REFERENCES `cocktails` (`id`),  
  ADD CONSTRAINT `cocktail_drinks_ibfk_4` FOREIGN KEY (`drink_id`)  
REFERENCES `drinks` (`id`);  
COMMIT;
```

2.3.4.16. A „cocktail_ingredients” tábla

Ez a tábla tartalmazza a koktélok összetevőit.

#	Név	Típus	Illesztés	Tulajdonságok	Nulla	Alapértelmezett	Megjegyzések	Extra
☐	1 id 	int(11)			Nem	Nincs		AUTO_INCREMENT
☐	2 cocktail_id 	int(11)			Nem	Nincs		
☐	3 ingredient_id 	int(11)			Nem	Nincs		
☐	4 quantity	int(11)			Nem	Nincs		

- *id*: azonosító, elsődleges kulcs
- *cocktail_id*: a koktél azonosítója
- *ingredient_id*: az összetevő azonosítója
- *quantity*: mennyiség

Tábla létrehozása:

```
CREATE TABLE `cocktail_ingredients` (  
  `id` int(11) NOT NULL,  
  `cocktail_id` int(11) NOT NULL,  
  `ingredient_id` int(11) NOT NULL,  
  `quantity` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4  
COLLATE=utf8mb4_general_ci;  
ALTER TABLE `cocktail_ingredients`  
  ADD PRIMARY KEY (`id`),  
  ADD KEY `cocktail_id` (`cocktail_id`),  
  ADD KEY `ingredient_id` (`ingredient_id`);  
ALTER TABLE `cocktail_ingredients`  
  MODIFY `id` int(11) NOT NULL AUTO_INCREMENT;  
ALTER TABLE `cocktail_ingredients`  
  ADD CONSTRAINT `cocktail_ingredients_ibfk_3` FOREIGN KEY  
  (`cocktail_id`) REFERENCES `cocktails` (`id`),  
  ADD CONSTRAINT `cocktail_ingredients_ibfk_4` FOREIGN KEY  
  (`ingredient_id`) REFERENCES `ingredients` (`id`);  
COMMIT;
```

2.4. Jogosultságok, jogosultsági szintek

Az asztali alkalmazásban nincsenek elkülönített szerepkörök. Ez a felület a dolgozók részére van kialakítva, aminek használatához nem szükséges regisztráció.

2.4.1. Nem regisztrált felhasználó

A rendszerhez való hozzáférés korlátozott és kizárólag olvasási hozzáférést biztosít a nem regisztrált felhasználók számára. A vendégfelhasználók megtekinthetik a digitális itallapot, azonban az interaktív és tranzakciós műveletek végrehajtására nincs jogosultságuk. Az alábbi funkciók elérése korlátozott számukra:

- asztalfoglalás az online felületen
- rendelés leadása az asztaloknál elhelyezett QR-kódokkal
- egyedi koktélok összeállítása
- korábbi rendelések és számlák megtekintése

Ezekhez a funkciókhoz a felhasználóknak regisztrálnia kell, majd be kell jelentkeznie a saját profiljába, biztosítva ezzel a személyre szabott kiszolgálást és a biztonságos fizetési folyamatokat.

2.4.2. Regisztrált felhasználó

A regisztrált felhasználók a Ponth elsődleges célcsoportját képezik, akik a bejelentkezést követően teljes körű hozzáférést kapnak az alkalmazás szolgáltatásaihoz. A regisztrálás az alábbi jogosultságokat biztosítja:

- asztalfoglalás kezelése: a felhasználók saját névre szóló foglalásokat indíthatnak, amelyeket a profiljukban bármikor nyomon követhetnek és módosíthatnak
- hozzáférés az asztali QR-kód alapú rendelési rendszerhez, amely lehetővé teszi a közvetlen fogyasztást és a digitális kosárkezelést
- lehetőség nyílik meglévő receptek módosítására vagy teljesen új, saját ízlés szerinti italkombináció összeállítására és elmentésére
- a korábbi vásárlások visszakereshetősége, a digitális számlák elérése

2.4.3.Admin

Az adminisztrátor a legmagasabb szintű jogosultságokkal rendelkező felhasználó, aki a rendszer minden eleméhez hozzáfér és a felhasználókat kezeli. Ez a jogosultság az alábbi műveletek kezeléséhez ad hozzáférést:

- meglévő felhasználók előléptetése adminisztrátori rangra, vagy a kiemelt jogosultságok visszavonása
- a szabályszegő vagy inaktív felhasználói profilok végleges törlése

2.5. API dokumentáció

2.5.1. OrderController

Az **OrderController** végpontjai a vendégek rendelési folyamatát, a digitális számlavezetést (Open Bill) és a fizetési tranzakciót kezelik. Az alábbiakban részletezzük az egyes végpontok feladatait, a várt adatokat és a visszatérési értékeket.

Számla ellenőrzése és nyitása (GET)

```
Route::post('/check-bill/{boxId}', [OrderController::class, 'checkBill']);
public function checkBill(Request $request, $boxId)
{
    $user = $request->user();

    $openBill = OpenBill::where('user_id', $user->id)->first();

    if (!$openBill) {
        // Create a new open bill
        $openBill = OpenBill::create([
            'user_id' => $user->id,
            'box_id' => $boxId,
            'totalSum' => 0,
        ]);

        return response()->json([
            'status' => 'opened',
            'bill' => $openBill->load('box'),
        ], 201);
    }

    if ($openBill->box_id == $boxId) {
        return response()->json([
            'status' => 'existing',
            'bill' => $this->billWithCocktailNames($openBill->load('box', 'orders.drink', 'orders.extras')),
        ]);
    }

    // Different box - unresolved bill
    $openBill->load('box');
    return response()->json([
        'error' => "You have an unresolved bill at Box {$openBill->box->number}. Please pay it first.",
        'bill' => $openBill,
    ], 409);
}
```

Az **api/check-bill/{boxId}** végponton keresztül a rendszer ellenőrzi, hogy a bejelentkezett felhasználónak van-e aktív fogyasztása az adott asztalnál. Amennyiben még nincs nyitott számlája, a rendszer létrehoz egy új OpenBill rekordot (201-es kód). Ha már van számlája ugyanannál az asztalnál, visszatér annak adataival és korábbi rendeléseivel. Ha a felhasználó egy másik asztalnál rendelkezik lezáratlan számlával, a végpont 409-es (Conflict) hibaüzenetet küld a korábbi asztal adataival.

Mobil rendelés leadása (POST)

```
Route::post('/mobile-order', [OrderController::class, 'mobileStore']);
public function mobileStore(Request $request)
{
    $request->validate([
        // ...
    ]);

    $user = $request->user();
    $openBill = OpenBill::where('user_id', $user->id)->first();

    if (!$openBill) {
        return response()->json([
            'error' => 'No open bill found. Please scan a QR code first.',
        ], 400);
    }

    return DB::transaction(function () use ($request, $openBill) {
        $orders = [];
        $totalAdded = 0;

        foreach ($request->items as $item) {
            // Custom cocktail order
            if (!empty($item['custom_cocktail_id'])) {
                $custom = \App\Models\CustomCocktail::findOrFail($item['custom_cocktail_id']);
                $subtotal = $custom->price * $item['quantity'];

                $order = Order::create([
                    // ...
                ]);

                // Store custom cocktail reference in orders_extra (type=' ' as marker)
                OrderExtra::create([
                    // ...
                ]);

                $orders[] = $order;
                $totalAdded += $subtotal;
                continue;
            }

            // Standard drink order
            $drink = Drink::findOrFail($item['drink_id']);
            $extrasCost = 0;

            // Calculate extras cost (only charge for positive deltas = added extras)
            if (!empty($item['extras'])) {
                foreach ($item['extras'] as $extra) {
                    if ($extra['delta'] > 0) {
                        if ($extra['type'] === 'Drink') {
                            $extraDrink = Drink::findOrFail($extra['item_id']);
                            $extrasCost += $extraDrink->mixing_price * $extra['delta'];
                        } else {
                            $ingredient = \App\Models\Ingredient::findOrFail($extra['item_id']);
                            $extrasCost += $ingredient->price * $extra['delta'];
                        }
                    }
                }
            }
        }
    });
}
```

```
$subtotal = ($drink->price + $extrasCost) * $item['quantity'];

$order = Order::create([
    // ...
]);

// Save extras to orders_extra
if (!empty($item['extras'])) {
    foreach ($item['extras'] as $extra) {
        if ($extra['delta'] != 0) {
            $price = 0;
            if ($extra['type'] === 'Drink') {
                $extraDrink = Drink::find($extra['item_id']);
                $price = $extraDrink ? $extraDrink->mixing_price * max(0, $extra['delta']) : 0;
            } else {
                $ingredient = \App\Models\Ingredient::find($extra['item_id']);
                $price = $ingredient ? $ingredient->price * max(0, $extra['delta']) : 0;
            }
            OrderExtra::create([
                // ...
            ]);
        }
    }
}

$orders[] = $order;
$totalAdded += $subtotal;

// Update the open bill total
$openBill->increment('totalSum', $totalAdded);

return response()->json([
    'message' => 'Order placed successfully!',
    'orders' => $orders,
    'billTotal' => $openBill->fresh()->totalSum,
], 201);
});
```

Az **api/mobile-order** végpont szolgál a rendelések rögzítésére az aktív számlán. A kérés törzsében egy items listát kell küldeni, amely tartalmazhat normál italokat (drink_id) vagy egyedi koktélok (custom_cocktail_id). A rendszer kiszámítja az alapárakat és az esetleges extrák (pl. plusz összetevők) költségeit, majd ezekkel növeli a számla végösszegét.

Aktuális nyitott számla lekérése (GET)

```
Route::get('/my-open-bill', [OrderController::class, 'myOpenBill']);
public function myOpenBill(Request $request)
{
    $user = $request->user();

    $openBill = OpenBill::with(['box', 'orders.drink', 'orders.extras'])
        ->where('user_id', $user->id)
        ->first();

    if (!$openBill) {
        return response()->json(['bill' => null]);
    }

    $this->resolveCustomCocktailNames($openBill->orders);

    return response()->json(['bill' => $openBill]);
}
```

Az **api/my-open-bill** végpont segítségével a felhasználó bármikor lekérdezheti a jelenleg futó fogyasztását. A kérés visszaadja az asztal adatait, a rendelt italok listáját (extrákkal együtt), és automatikusan feloldja az egyedi koktélok elnevezéseit. Ha a felhasználónak nincs aktív számlája, a válasz egy üres bill objektummal tér vissza.

Számla lezárása és fizetés (POST)

```
Route::post('/checkout', [OrderController::class, 'checkout']);
0 references | 0 implementations
class CustomCocktailController extends Controller {
    0 references | 0 overrides
    public function checkout(Request $request)
    {
        $request->validate([
            'paid_with' => 'required|in:Card,Cash',
        ]);

        $user = $request->user();
        $openBill = OpenBill::where('user_id', $user->id)->first();

        if (!$openBill) {
            return response()->json([
                'error' => 'No open bill to pay.',
            ], 400);
        }

        return DB::transaction(function () use ($request, $user, $openBill) {
            // Calculate actual total from orders
            $total = Order::where('bills_id', $openBill->id)->sum('subtotal');

            // Create the archived bill
            $bill = Bill::create([
                'box_id' => $openBill->box_id,
                'user_id' => $user->id,
                'total' => $total,
                'paid_at' => now(),
                'paid_with' => $request->paid_with,
            ]);

            // Link orders to the paid bill and set status
            Order::where('bills_id', $openBill->id)
                ->update([
                    'paid_bill_id' => $bill->id,
                    'status' => 'served',
                ]);

            // Delete the open bill
            $openBill->delete();

            return response()->json([
                'message' => 'Bill paid successfully!',
                'bill' => $bill->load('box'),
            ]);
        });
    }
}
```

Az **api/checkout** végponton keresztül történik a fogyasztás véglegesítése. A kérésben megadott fizetési mód (Card vagy Cash) rögzítése után a rendszer a rendeléseket archiválja a Bill táblába, a tételek státuszát „kiszolgáltra” állítja, az ideiglenes nyitott számlát pedig törli az adatbázisból.

Fizetési előzmények lekérése (GET)

```
Route::get('/my-bills', [OrderController::class, 'myBills']);
0 references | 0 implementations
class CustomCocktailController extends Controller {
    0 references | 0 overrides
    public function myBills(Request $request)
    {
        $user = $request->user();

        $bills = Bill::with(['box', 'orders.drink', 'orders.extras'])
            ->where('user_id', $user->id)
            ->orderBy('paid_at', 'desc')
            ->get();

        foreach ($bills as $bill) {
            $this->resolveCustomCocktailNames($bill->orders);
        }

        return response()->json($bills);
    }
}
```

Az **api/my-bills** végponton a felhasználó elérheti az összes korábbi, már kifizetett számláját. Az adatok csökkenő időrendi sorrendben érkeznek, így a felhasználó nyomon követheti korábbi fogyasztásait, a fizetett összegeket és a rendelt tételek részleteit.

Hagyományos rendelés rögzítés (POST)

```
Route::post('/orders', [OrderController::class, 'store']);
public function store(Request $request)
{
    $request->validate([
        'box_detail_id' => 'required|exists:boxes_details,id',
        'item_id' => 'required|exists:drinks,id',
        'quantity' => 'required|integer|min:1'
    ]);

    return DB::transaction(function () use ($request) {
        $drink = Drink::findOrFail($request->item_id);
        $subtotal = $drink->price * $request->quantity;

        $order = Order::updateOrCreate(
            ['box_detail_id' => $request->box_detail_id, 'item_id' => $request->item_id],
            ['quantity' => DB::raw("quantity + $request->quantity"), 'subtotal' => DB::raw("subtotal + $subtotal")]
        );

        return response()->json($order, 201);
    });
}
```

Az **api/orders** végpont egy egyszerűsített, úgynevezett "legacy" rendelési felületet biztosít. Elsősorban olyan esetekben használatos, ahol nincs szükség extra összetevők vagy egyedi koktélok kezelésére. A rendszer az asztal azonosítója és a termék azonosítója alapján megkeresi a tételt: ha már létezik, növeli annak mennyiségét és részösszegét, ha nem létezik, új rekordot hoz létre.

Egyedi koktélnévek feloldása (Belső logika)

```
private function resolveCustomCocktailNames($orders)
{
    $ccIds = [];
    foreach ($orders as $order) {
        if ($order->item_id == 1) {
            $marker = $order->extras->firstWhere('type', '');
            if ($marker) {
                $ccIds[] = $marker->item_id;
            }
        }
    }

    if (empty($ccIds)) return;

    $names = CustomCocktail::whereIn('id', $ccIds)->pluck('cocktail_name', 'id');

    foreach ($orders as $order) {
        if ($order->item_id == 1) {
            $marker = $order->extras->firstWhere('type', '');
            if ($marker && isset($names[$marker->item_id])) {
                $order->setAttribute('custom_cocktail_name', $names[$marker->item_id]);
            }
        }
    }
}
```

Mivel az adatbázis-struktúra az egyedi koktélokot egy általános helykitöltő azonosítóval (item_id: 1) tárolja, a vezérlő tartalmaz egy belső segédfüggvényt a nevek feloldására. Ez a logika az orders_extra táblában tárolt referenciák alapján kikeresi a koktélok valódi elnevezését a CustomCocktail modellből, majd dinamikusan hozzáadja azt a válaszbjektumhoz custom_cocktail_name néven. Ez biztosítja, hogy a felhasználói felületen a technikai azonosítók helyett a vendég által adott fantázianév jelenjen meg.

Számla formázása névfeloldással (Belső segédfüggvény)

```
private function billWithCocktailNames($bill)
{
    $this->resolveCustomCocktailNames($bill->orders);
    return $bill;
}
```

Ez a segédfüggvény a kódismétlés elkerülése érdekében jött létre. Feladata, hogy egy adott számla objektum összes rendelési tételén lefuttassa a névfeloldó logikát, mielőtt a rendszer visszaküldené azt a kliensoldalnak. Ez garantálja a konzisztens adatmegjelenítést mind a számlanyitáskor, mind a lekérdezésekkor.

2.5.2. CustomCocktailController

A CustomCocktailController végpontjai a felhasználók által összeállított egyedi koktélok kezeléséért felelnek. Lehetővé teszik a saját receptek mentését, módosítását, törlését, valamint az automatikus árkalkulációt az összetevők alapján.

Saját egyedi koktélok lekérése (GET)

```
Route::get('/my-custom-cocktails', [CustomCocktailController::class, 'index']);
1 reference | 0 implementations
class CustomCocktailController extends Controller {
1 reference | 0 overrides
public function index(Request $request) {
    return $request->user()
        ->customCocktails()
        ->with(['drinks.mixingQuantity', 'ingredients.quantityUnit'])
        ->get();
}
```

Az **api/my-custom-cocktails** végponton keresztül a bejelentkezett felhasználó lekérdezheti az összes korábban elmentett egyedi receptjét. A válasz tartalmazza a koktélok nevét, a kalkulált árat, valamint a hozzájuk tartozó italok és egyéb összetevők (pl. díszítés, szirup) pontos listáját és mennyiségi egységeit.

Egyedi koktél létrehozása (POST)

```
Route::apiResource('custom-cocktails', CustomCocktailController::class);
1 reference | 0 implementations
class CustomCocktailController extends Controller {
0 references | 0 overrides
    public function store(Request $request) {
        $request->validate([
            'cocktail_name' => [
                'required', 'string', 'max:100',
                Rule::unique('custom_cocktail')->where('user_id', $request->user()->id)
            ],
            'drinks' => 'sometimes|array',
            'drinks.*.id' => 'required_with:drinks|exists:drinks,id',
            'drinks.*.quantity' => 'required_with:drinks|integer|min:1',
            'ingredients' => 'sometimes|array',
            'ingredients.*.id' => 'required_with:ingredients|exists:ingredients,id',
            'ingredients.*.quantity' => 'required_with:ingredients|integer|min:1',
        ]);

        return DB::transaction(function () use ($request) {
            // Calculate price
            $price = 0;

            $drinkSync = [];
            if ($request->has('drinks')) {
                foreach ($request->drinks as $d) {
                    $drink = Drink::findOrFail($d['id']);
                    $price += $drink->mixing_price * $d['quantity'];
                    $drinkSync[$d['id']] = ['quantity' => $d['quantity']];
                }
            }

            $ingredientSync = [];
            if ($request->has('ingredients')) {
                foreach ($request->ingredients as $i) {
                    $ingredient = Ingredient::findOrFail($i['id']);
                    $price += $ingredient->price * $i['quantity'];
                    $ingredientSync[$i['id']] = ['quantity' => $i['quantity']];
                }
            }

            $custom = $request->user()->customCocktails()->create([
                'cocktail_name' => $request->cocktail_name,
                'price' => $price,
            ]);

            $custom->drinks()->sync($drinkSync);
            $custom->ingredients()->sync($ingredientSync);

            return response()->json(
                $custom->load(['drinks.mixingQuantity', 'ingredients.quantityUnit']),
                201
            );
        });
    }
}
```

Az **api/custom-cocktails** végponton (POST metódussal) hozható létre új recept. A rendszer validálja, hogy a név egyedi-e a felhasználó saját receptjei között. A mentés során a vezérlő automatikusan kiszámítja a koktél végleges árát az összetevők alapárai és mennyiségei alapján, majd rögzíti a kapcsolatokat a kiegészítő táblákban.

Egyedi koktélok módosítása (PUT)

```
Route::apiResource('custom-cocktails', CustomCocktailController::class);
1 reference | 0 implementations
class CustomCocktailController extends Controller {
0 references | 0 overrides
    public function update(Request $request, $id) {
        $custom = CustomCocktail::findOrFail($id);

        if ($custom->user_id !== $request->user()->id) {
            return response()->json(['error' => 'Unauthorized'], 403);
        }

        $request->validate([
            'cocktail_name' => [
                'sometimes', 'string', 'max:100',
                Rule::unique('custom_cocktail')->where('user_id', $request->user()->id)->ignore($id)
            ],
            'drinks' => 'sometimes|array',
            'drinks.*.id' => 'required_with:drinks|exists:drinks,id',
            'drinks.*.quantity' => 'required_with:drinks|integer|min:1',
            'ingredients' => 'sometimes|array',
            'ingredients.*.id' => 'required_with:ingredients|exists:ingredients,id',
            'ingredients.*.quantity' => 'required_with:ingredients|integer|min:1',
        ]);

        return DB::transaction(function () use ($request, $custom) {
            if ($request->has('cocktail_name')) {
                $custom->cocktail_name = $request->cocktail_name;
            }

            // Recalculate price
            $price = 0;

            if ($request->has('drinks')) {
                $drinkSync = [];
                foreach ($request->drinks as $d) {
                    $drink = Drink::findOrFail($d['id']);
                    $price += $drink->mixing_price * $d['quantity'];
                    $drinkSync[$d['id']] = ['quantity' => $d['quantity']];
                }
                $custom->drinks()->sync($drinkSync);
            } else {
                // Keep existing drinks in price
                foreach ($custom->drinks as $d) {
                    $price += $d->mixing_price * $d->pivot->quantity;
                }
            }

            if ($request->has('ingredients')) {
                $ingredientSync = [];
                foreach ($request->ingredients as $i) {
                    $ingredient = Ingredient::findOrFail($i['id']);
                    $price += $ingredient->price * $i['quantity'];
                    $ingredientSync[$i['id']] = ['quantity' => $i['quantity']];
                }
                $custom->ingredients()->sync($ingredientSync);
            } else {
                foreach ($custom->ingredients as $i) {
                    $price += $i->price * $i->pivot->quantity;
                }
            }

            $custom->price = $price;
            $custom->save();

            return response()->json(
                $custom->load(['drinks.mixingQuantity', 'ingredients.quantityUnit'])
            );
        });
    }
}
```

Az `api/custom-cocktails/{id}` végponton keresztül (PUT vagy PATCH metódussal) módosítható egy már létező recept. A rendszer ellenőrzi a jogosultságot, így a felhasználó csak a saját receptjeit szerkesztheti. Amennyiben változnak az összetevők vagy azok mennyisége, a vezérlő újra kalkulálja a koktél árát.

Egyedi koktél törlése (DELETE)

```
Route::apiResource('custom-cocktails', CustomCocktailController::class);
1 reference | 0 implementations
class CustomCocktailController extends Controller {
    0 references | 0 overrides
    public function destroy($id, Request $request) {
        $item = CustomCocktail::findOrFail($id);
        if ($item->user_id !== $request->user()->id) {
            return response()->json(['error' => 'Unauthorized'], 403);
        }
        $item->drinks()->detach();
        $item->ingredients()->detach();
        $item->delete();
        return response()->json(null, 204);
    }
}
```

Az **api/custom-cocktails/{id}** végponton keresztül (DELETE metódussal) távolítható el egy recept a rendszerből. A törlés előtt a rendszer automatikusan felbontja a kapcsolatokat a recepthoz rendelt italokkal és összetevőkkel (detach), majd véglegesen törli a koktét az adatbázisból. Sikeres művelet esetén 204-es (No Content) válasz érkezik.

2.6. A projekt tesztelése

A projekt fejlesztése során a tesztelés nem egy lezáró szakaszként, hanem a kódolással párhuzamosan végzett tevékenységként jelent meg. A stratégia alapját a többszintű ellenőrzés adta, amely a forráskód automatizált vizsgálatától a manuális felhasználói próbákig terjedt.

2.6.1. Tesztelés és minőségbiztosítás

A fejlesztési ciklus során a minőségellenőrzés nem egy elkülönített, záró fázisként, hanem a kódolással szorosan integrált, iteratív folyamatként valósult meg. A szoftver megbízhatóságát egy többszintű tesztelési stratégia garantálta, amely az automatizált forráskód-analízistől a szimulált felhasználói forgatókönyvek manuális végrehajtásáig terjedt.

2.6.1.1. A készre jelentés kritériumai (Definition of Done):

Az alkalmazást akkor tekintettük készre jelenthetőnek és a vizsgakövetelményeknek megfelelőnek, amikor az alábbi minőségi elvárások maradéktalanul teljesültek:

- **Platformfüggetlen megjelenés:** A webes interfész és az asztali kliens is konzisztens felhasználói élményt nyújt különböző képernyőfelbontások és eszközök mellett.
- **Kalkulációs precizitás:** Az alapanyag-nyilvántartás és a pénzügyi elszámolás (árazás, tételek összegének kiszámítása) minden tesztelésben matematikai pontossággal működik.
- **Funkcionális integritás:** A projekttervben rögzített összes szolgáltatás (az asztalfoglalástól a QR-kódos rendelésig) a specifikációnak megfelelően üzemel.
- **Üzembiztonság:** A rendszer mentes az olyan kritikus programhibáktól, amelyek adatvesztést vagy a futási környezet váratlan leállítását okoznák.

2.6.1.2. Alkalmazott minőségbiztosítási technikák

A fejlesztés alatt az alábbi módszertanokat hívtuk segítségül a hibák korai feltárása érdekében:

- **Kódstílus és statikus vizsgálat:** A Visual Studio fejlesztőkörnyezet analízáló eszközeivel folyamatosan monitoroztuk a forráskód tisztaságát. Ez segített a logikai ellentmondások és a felesleges memóriefoglalások eliminálásában még a fordítási szakaszban.
- **Automatizált egységtesztek (Unit Test):** A kritikus üzleti logikát – különös tekintettel a regisztráció folyamatára és az árazási algoritmusokra – elkülönített tesztesetekkel validáltuk. Ezek a vizsgálatok biztosítják, hogy a kód későbbi bővítése ne okozzon hibát a már stabil funkciókban.
- **Dinamikus hibakeresés (Debugging):** A futás idejű elemzések során töréspontok segítségével követtük nyomon az adatok útját az adatbázis és a felhasználói felület között, külön figyelmet fordítva az SQL lekérdezések hatékonyságára.

2.6.1.3. Egység tesztek (Unit Tests)

Az alkalmazás logikájának ellenőrzése automatizált tesztek segítségével történt. A tesztkörnyezet a Laravel beépített tesztelési keretrendszerére (PHPUnit) épül, amely lehetővé teszi az adatbázis-műveletek és a http kérések izolált vizsgálatát. A tesztesetek minden esetben az Arrange-Act-Assert (Előkészítés-Végrehajtás-Ellenőrzés) mintát követik.

Felhasználói szolgáltatások és hozzáférés-kezelés ellenőrzése

A UserController tesztelése során a regisztrációs folyamat stabilitása, a biztonsági protokollok és az adminisztrátori jogosultságok érvényesítése állt a középpontban. A vizsgálat célja annak igazolása, hogy a rendszer képes a beérkező adatok alapján biztonságos felhasználói profilokat létrehozni és kezelni.

A vizsgált logikai pontok:

- **Regisztrációs validáció és jelszókezelés:** Annak ellenőrzése, hogy a rendszer megfelelően kezeli-e a kötelező mezőket és az egyedi e-mail címeket. A teszt igazolja, hogy a jelszavak a *Hash::make* eljárással titkosítva kerülnek az adatbázisba, és a sikeres regisztráció után a *Sanctum* token generálása megtörténik.
- **Biztonsági korlátok és Pénzügyi védelem:** A törlési folyamat (*deleteUser*) kritikus tesztelése során igazoltuk, hogy a szoftver megakadályozza a felhasználó eltávolítását, amennyiben az *open_bills* táblában aktív, rendezetlen számlája található. Ez a logikai gát védi a bizsrtót az esetleges anyagi veszteségektől.

```
/**
 * Regisztráció validációjának és a jelszó titkosításának tesztelése.
 */
0 references | 0 overrides
public function test_successful_registration_and_password_hashing()
{
    $response = $this->postJson('/api/register', [
        'name' => 'Teszt Felhasználó',
        'email' => 'teszt@gmail.com',
        'password' => '123456',
        'password_confirmation' => '123456',
    ]);

    $response->assertStatus(201)
        ->assertJsonStructure(['access_token', 'user']);

    $this->assertDatabaseHas('users', ['email' => 'teszt@gmail.hu']);
}
```

```

/**
 * Pénzügyi biztonsági korlát: Felhasználó nem törölhető nyitott számlával.
 */
0 references | 0 overrides
public function test_cannot_delete_user_with_open_bill()
{
    $admin = User::factory()->create(['is_admin' => true]);
    $user = User::factory()->create();

    // Nyitott számla rögzítése az adatbázisban
    DB::table('open_bills')->insert([
        'user_id' => $user->id,
        'totalSum' => 1200,
        'box_id' => 1
    ]);

    $response = $this->actingAs($admin)
        ->deleteJson("/api/users/{$user->id}");

    $response->assertStatus(409)
        ->assertJson(['error' => 'open_invoices']);
}

```

Egyedi koktélkezelési szolgáltatások ellenőrzése

A *CustomCocktailController* tesztelése során az egyedi receptúrák mentése, az automatikus árkalkuláció és a felhasználói jogosultságok érvényesítése állt a középpontban. A vizsgálat célja annak igazolása, hogy a rendszer képes a különböző összetevőkből konzisztens receptet és pontos végösszeget generálni.

A vizsgált logikai pontok:

- **Automatikus árkalkuláció:** Annak ellenőrzése, hogy az egyedi koktél létrehozásakor (store) a rendszer helyesen adja-e össze a felhasznált italok (mixing_price) és összetevők (price) egységárát a megadott mennyiségek alapján.
- **Jogosultságkezelés (Authorization):** Annak igazolása, hogy egy felhasználó csak a saját maga által készített koktélokot tudja módosítani (update) vagy törölni (destroy). A teszt 403-as hibát (Unauthorized) vár, ha egy felhasználó idegen *custom_cocktail_id*-val próbál műveletet végezni.

```

/**
 * Egyedi koktél létrehozásának és az áralkulációnak a tesztelése.
 */
0 references | 0 overrides
public function test_custom_cocktail_creation_and_price_calculation()
{
    $user = User::factory()->create();
    $drink = Drink::create(['name' => 'Vodka', 'mixing_price' => 500]);
    $ingredient = Ingredient::create(['name' => 'Lime', 'price' => 150]);

    $response = $this->actingAs($user)->postJson('/api/custom-cocktails', [
        'cocktail_name' => 'Teszt Koktél',
        'drinks' => [['id' => $drink->id, 'quantity' => 2]], // 2 * 500 = 1000
        'ingredients' => [['id' => $ingredient->id, 'quantity' => 1]] // 1 * 150 = 150
    ]);

    $response->assertStatus(201);
    // Elvárt ár: 1150
    $this->assertDatabaseHas('custom_cocktail', [
        'cocktail_name' => 'Teszt Koktél',
        'price' => 1150,
        'user_id' => $user->id
    ]);
}

```

```

/**
 * Idegen koktél törlésének megakadályozása.
 */
0 references | 0 overrides
public function test_cannot_delete_someone_elses_cocktail()
{
    $user1 = User::factory()->create();
    $user2 = User::factory()->create();
    $cocktail = CustomCocktail::create([
        'cocktail_name' => 'User1 Itala',
        'user_id' => $user1->id,
        'price' => 1000
    ]);

    $response = $this->actingAs($user2)->deleteJson("/api/custom-cocktails/{$cocktail->id}");

    $response->assertStatus(403);
}

```

C# - Cart logika

Az asztali alkalmazás alapját képező rendelési logika validálása során a Cart (Kosár) osztály metódusait vetettük alá egységteszteknek. Mivel a szoftver lehetővé teszi az italok egyedi módosítását, kiemelten fontos volt az összetett árazási struktúra (alapár + extrák összege szorozva a mennyiséggel) matematikai ellenőrzése.

A vizsgált logikai pontok:

- **Árkalkulációs hierarchia:** Annak igazolása, hogy a *CartItem.TotalItemPrice* helyesen összesíti az alapárat az extrák árával (*ExtrasPrice*), majd ezt megfelelően szorozza a mennyiséggel.

- **Egyedi azonosítók kezelése:** A rendszernek minden extrákkal ellátott italt külön tételként kell kezelnie (negatív kulcsgenerálás), míg az azonos típusú, extra nélküli italokat össze kell vonnia.
- **Mennyiségi műveletek:** A *RemoveOne* metódus ellenőrzése, különös tekintettel arra az esetre, amikor az utolsó darab eltávolítása után a tételnek törlődnie kell a kosárból.

```
public void TotalItemPrice_CalculatesCorrectlyWithMultipleExtras()
{
    // Arrange - Egy tétel összeállítása 2x mennyiséggel és extrákkal
    var item = new CartItem
    {
        Name = "Gin Tonic",
        Price = 1500,
        Quantity = 2,
        Extras = new List<CartExtra>
        {
            new CartExtra { Name = "Extra Gin", Price = 500, Quantity = 1 },
            new CartExtra { Name = "Lime", Price = 100, Quantity = 2 }
        }
    };

    // Act & Assert
    // ExtrasPrice: (500*1) + (100*2) = 700 Ft
    // TotalItemPrice: (1500 + 700) * 2 = 4400 Ft
    Assert.Equal(700, item.ExtrasPrice);
    Assert.Equal(4400, item.TotalItemPrice);
}
```

```
public void AddItemWithExtras_CreatesUniqueEntries()
{
    // Arrange
    var cart = new Cart();
    var extras = new List<CartExtra> { new CartExtra { Name = "Jég", Price = 0, Quantity = 1 } };

    // Act - Kétszer adjuk hozzá ugyanazt az italt ugyanazzal az extrával
    cart.AddItemWithExtras(1, "Whisky", 1200, "Drink", extras);
    cart.AddItemWithExtras(1, "Whisky", 1200, "Drink", extras);

    // Assert - Mivel van extra, két külön tételként kell szerepelniük (Count = 2)
    Assert.Equal(2, cart.Items.Count);
    Assert.Equal(2400, cart.TotalPrice());
}
```

```

public void RemoveOne_DecreasesQuantityOrRemovesItem()
{
    // Arrange
    var cart = new Cart();
    cart.AddItem(1, "Sör", 800, "Drink");
    cart.AddItem(1, "Sör", 800, "Drink"); // Quantity most 2

    // Act
    cart.RemoveOne(1);

    // Assert - Még bent kell lennie, de csak 1 darabbal
    Assert.Equal(1, cart.Items.First().Quantity);

    cart.RemoveOne(1);
    // Assert - Most már üresnek kell lennie
    Assert.Empty(cart.Items);
}

```

3. Felhasználói dokumentáció

3.1. Rendszerkövetelmény

3.1.1. Szoftveres feltételek

- **Operációs rendszer:** Windows 10/11, macOS, Linux, Android/iOS (az oldal reszponzivitása miatt)
- **Támogatott böngészők:** Google Chrome, Mozilla Firefox, Microsoft Edge, Safari

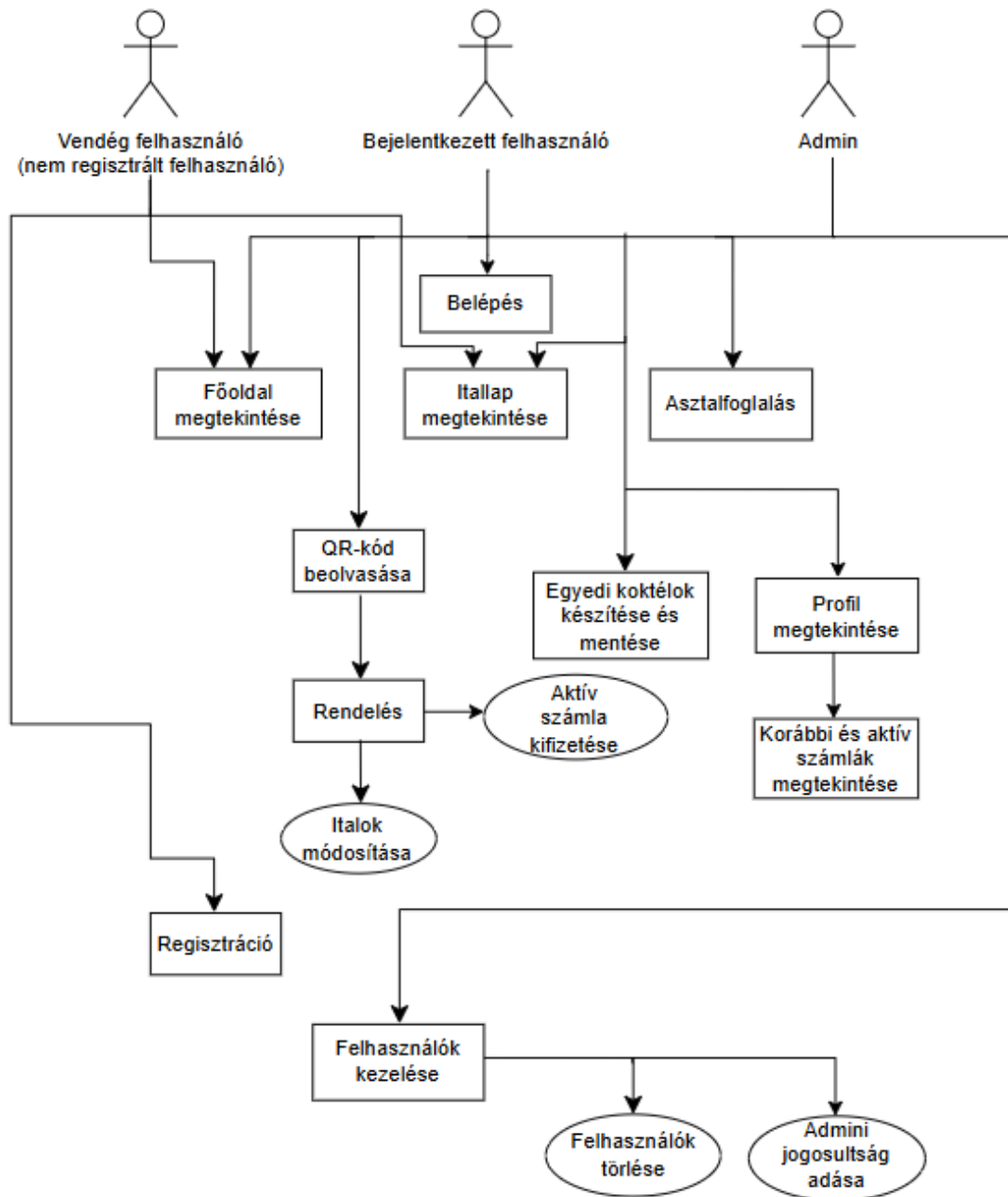
3.1.2. Hardveres feltételek

- **Processzor:** Bármilyen modern kétmagos processzor
- **Memória (RAM):** minimum 2 GB RAM
- **Beviteli eszköz:** Billentyűzet és egér, vagy érintőképernyő, olyan eszköz, aminek van kamerája

3.1.3. Hálózati feltételek

- **Internetkapcsolat:** Folyamatos szélessávú internetkapcsolat szükséges

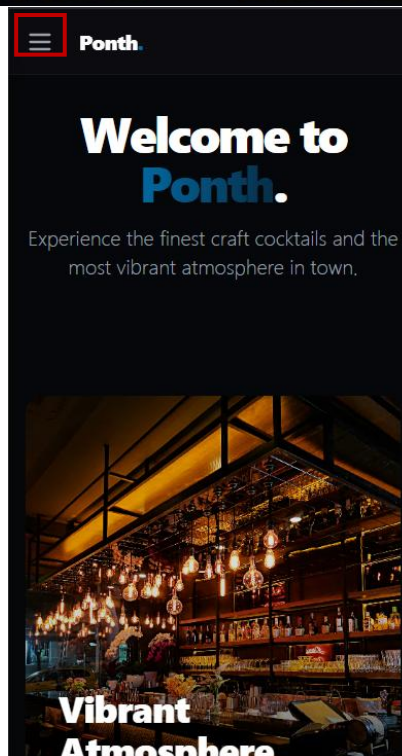
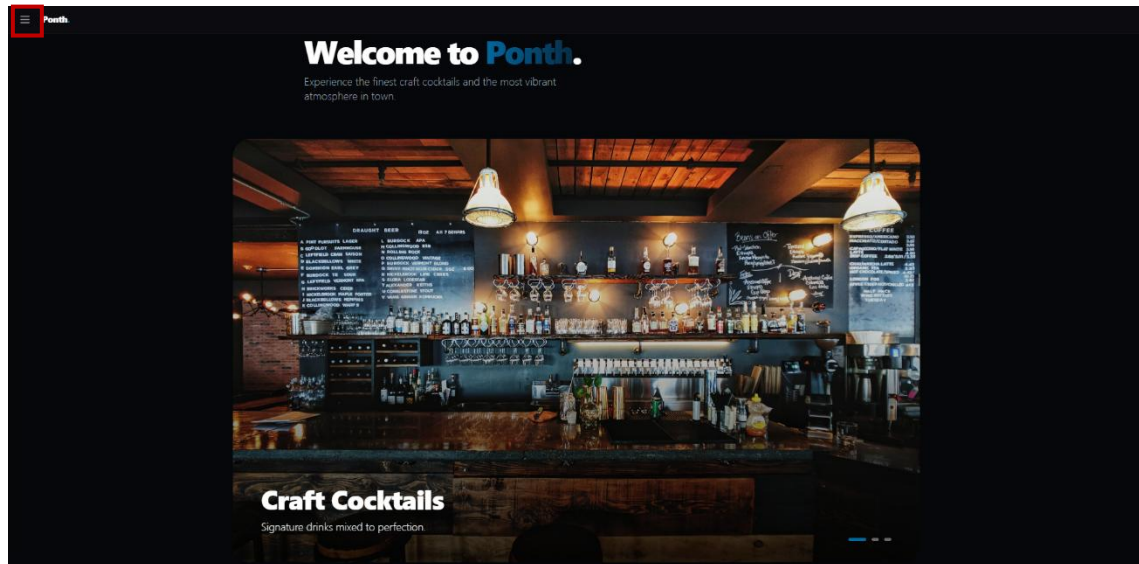
3.2. Használati eset diagram



3.3. Felhasználói felület

3.4. Weboldal bemutatása

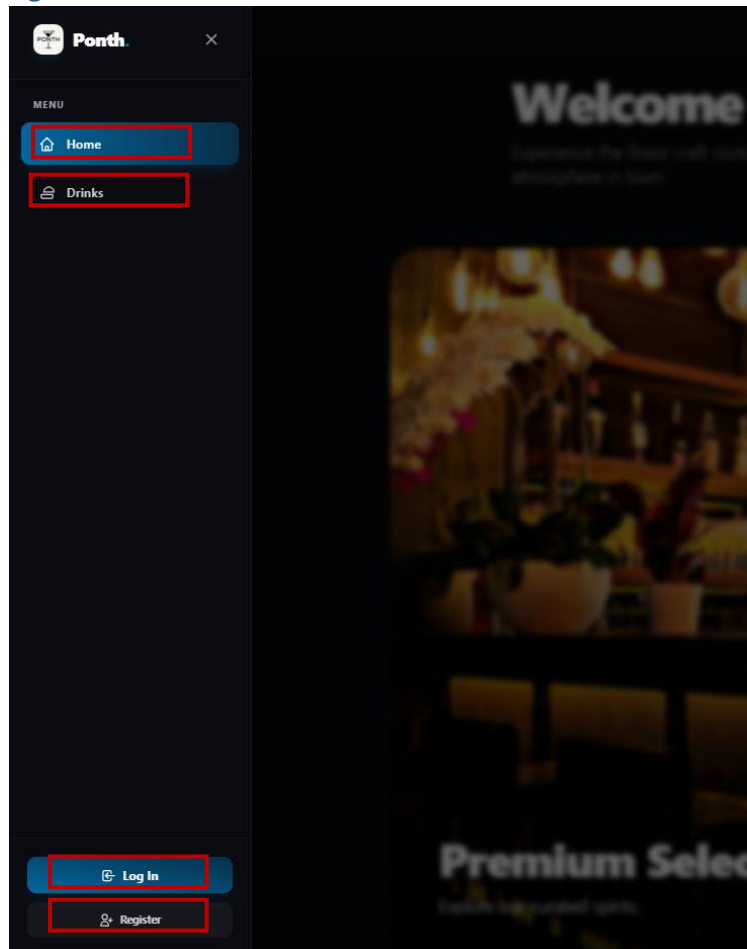
3.4.1. Üdvözlő oldal



Miután a felhasználó elindította az alkalmazást, az üdvözlő oldalra érkezik. A design a modern UI/UX irányelveket követi: a sötét tónusú háttér és a kontrasztos akció gombok segítik a fókusz megtartását. A felület reszponzív, így mobilkészüléken és táblagépen is optimális élményt nyújt. Az ikonra kattintva

lehet a szélső navigációs menüt elérni. mobilkészüléken és táblagépen is optimális élményt nyújt. Az  ikonra kattintva lehet a szélső navigációs menüt elérni.

3.4.2. Navigációs sáv

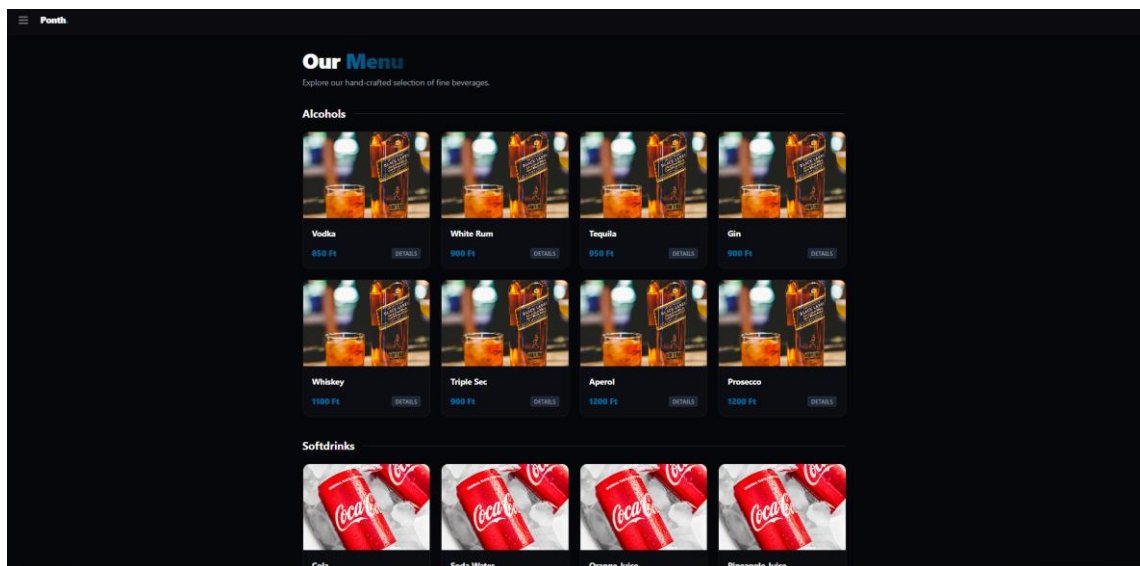


Ezen a felületen tudnak a felhasználók váltani az oldalak között.

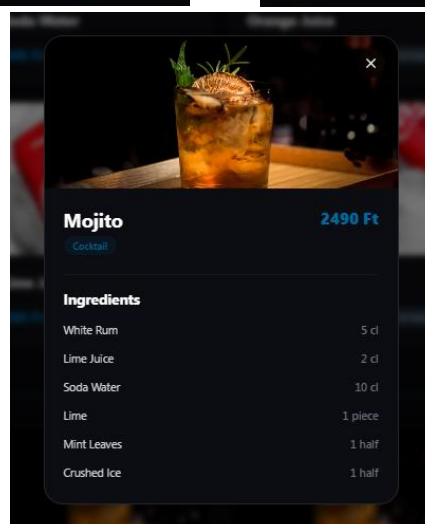
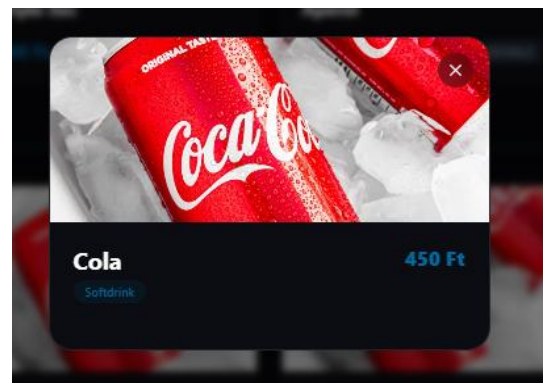
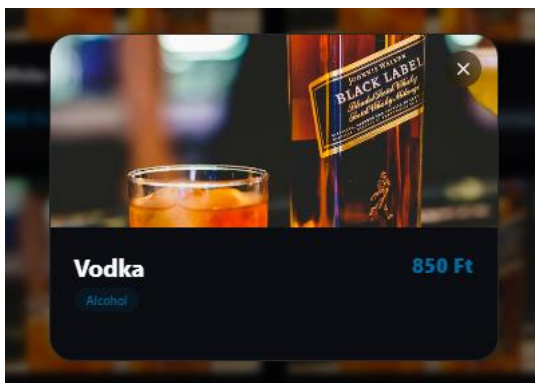
Innen érik el a(z):

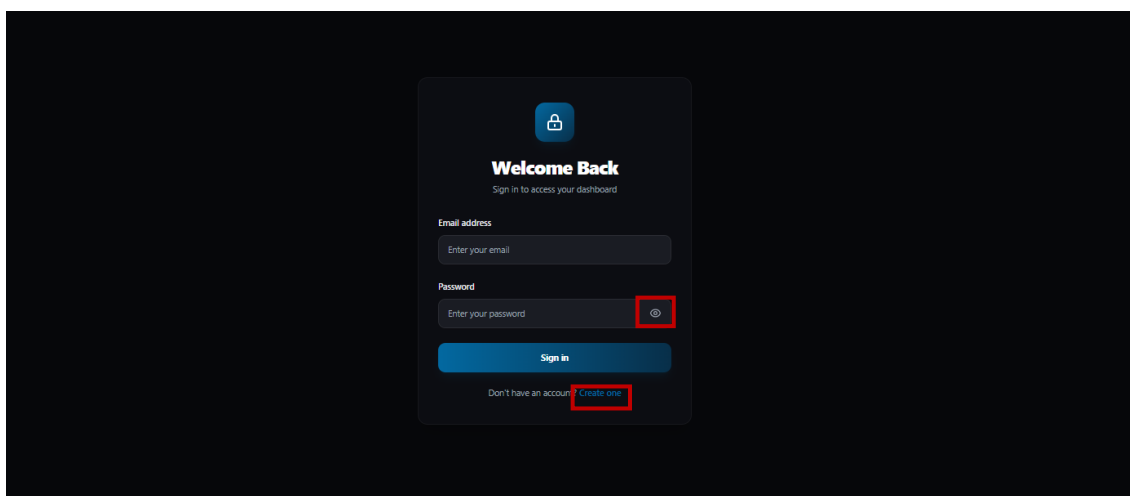
- **Főoldal (Home)**
- **Itallap (Drinks):** A bárban elérhető italok.
- **Bejelentkezés (Log In):** Bejelentkezési felület.
- **Regisztráció (Register):** Regisztrációs felület.

3.4.3. Itallap – Drinks



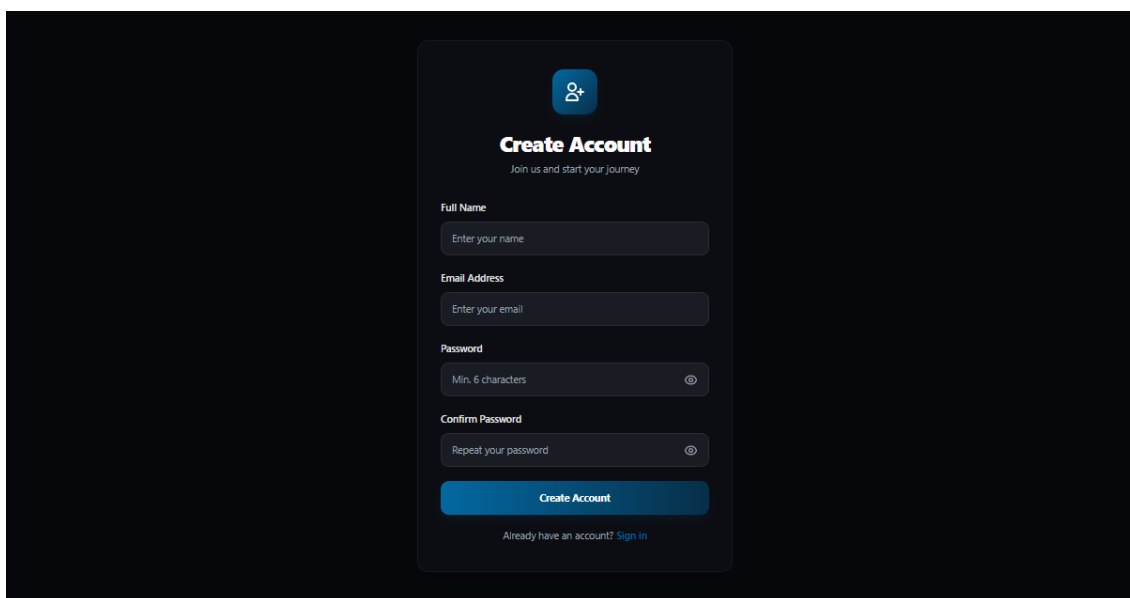
Ezen a felületen tekinthetők meg az elérhető italok és koktélok. Itt láthatóak az egyes italok/koktélok nevei és árai. Az egyes kártyákra kattintva érhető el az italok/koktélok részletei. Az italoknál a név, ár és a típusa látható (alcohol, softdrink, cocktail), míg a koktéloknál láthatóak a pontos összetevők és azok mennyiségei is. Az itallap képeit a testreszabhatóság miatt, a bár/pub a saját környezetében készült képekkel vagy általuk kiválasztottakkal töltheti fel.





3.4.4. Bejelentkezés – Log In

Itt van lehetőség a regisztrált felhasználóknak a bejelentkezésre.  A felhasználók a korábban megadott e-mail címmel és jelszóval tudnak bejelentkezni. A jelszó titkosítva jelenik meg, de lehetőség van a megtekintésére a gombra kattintva. Ha egy felhasználó nem rendelkezik profillal, erről a felületről is elérhető a regisztrációs felület, a „Create one” linkre kattintva.

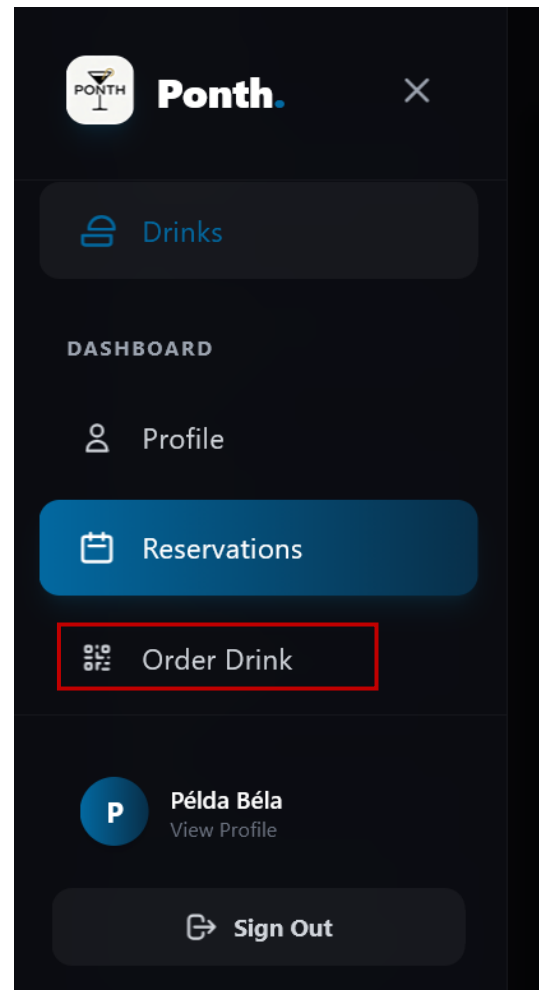
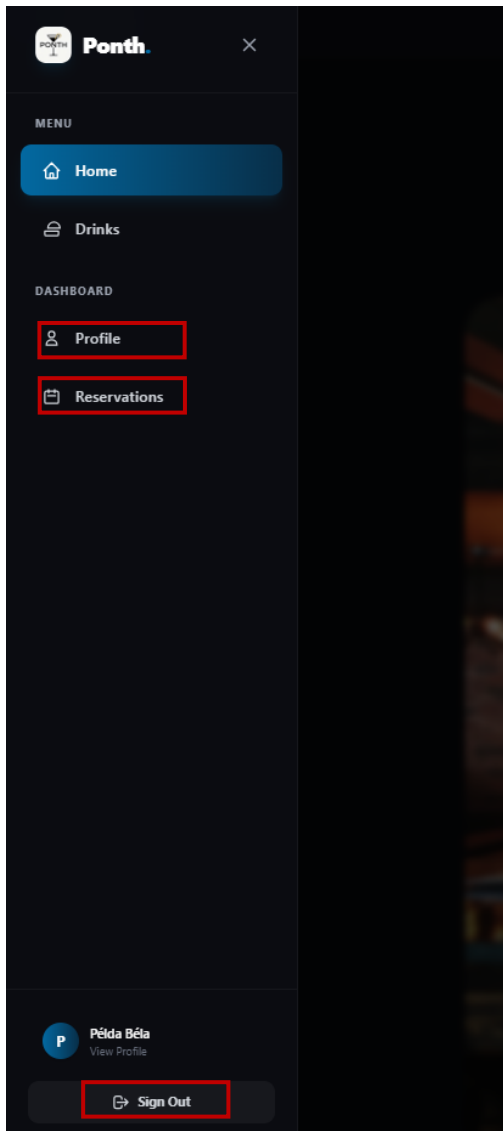


3.4.5. Regisztráció – Register

Itt van lehetőség a regisztrációra. A regisztráció során a felhasználónak meg kell adnia a teljes nevét, e-mail címét, és az általa választott jelszót kétszer, ezzel ellenőrizve, a jelszó pontosságát. Itt is van lehetőség a jelszavak megtekintésére. Az e-mail mező ellenőrzi, hogy a megadott e-mail megfelel-e az elvárt

formátumnak (pelda@pelda). A jelszó megadásánál is van elvárt formátum, a jelszónak 6 karakter hosszúnak kell minimum lennie. Regisztráció után már nem kell bejelentkezni, azonnal a fiókunkba dob.

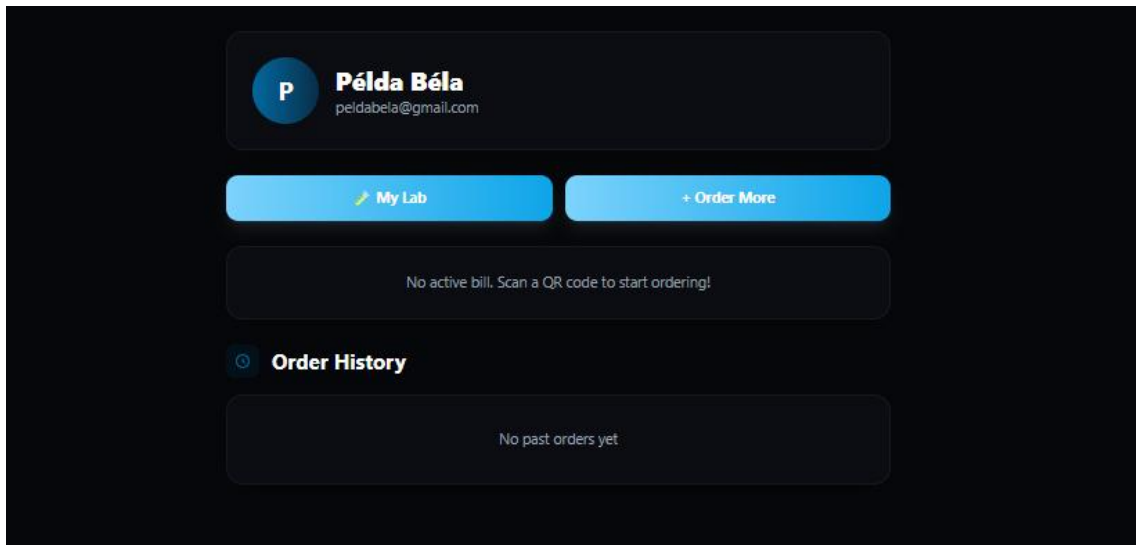
3.4.6. Bejelentkezett felhasználó



Bejelentkezés után az szélső navigációs menüben megjelennek további opciók. Bejelentkezett felhasználóként elérhető a személy profilja (Profile vagy Példa Béla) és az asztalfoglalás gombja is. Valamint a kijelentkezés lehetőség.

Mivel a rendelés csak egy QR-kód beolvasása után érhető el, így a rendelés lehetőség (Order Drink) csak akkor jelenik meg ha mobileszközön futtatjuk a szoftvert.

3.4.6.1. Profil



A profil felületen tekintheti meg a felhasználó éppen aktív számláját, illetve a már lezárt, kifizetett korábbi rendeléseit. Található a felületen két gomb. Az egyikkel érhető el a saját koktélok készítésének felülete (My Lab). Az Order More, pedig a navigációs menüben, mobileszközön megjelenő Order Drink felületre visz, ahol egy QR-kód beolvasása után van lehetőség a rendelésre.

3.4.6.2. Foglalás – Reservations

Reserve a Box
Book your private box for a perfect evening

Reservation created successfully!

New Reservation

Select Box

Box 1
4 seats

Box 2
4 seats

Box 3
2 seats

Box 4
5 seats

Box 5
4 seats

Date
yyyy. hh. nn.

Time
Select time...

Duration
1 hour

Name
Példa Béla

Phone
+36

Reserve Box

Upcoming Reservations
Active bookings

1 Box 1 - Példa Béla
2025-04-17 at 21:00:00 - 90 min
+36244354466
Reserved X

1 active reservations

Ez a felület a navigációs menü Reservations gombjával érhető el, bejelentkezett felhasználóknak. Itt tudja a felhasználó megadni a foglalás részleteit. Kiválaszthatja melyik asztalhoz szeretne ülni, az asztaloknál meg van jelenítve, hogy melyik hány főre alkalmas. Ezt követően a felhasználó kiválaszthatja melyik napon és melyik időpontban szeretne foglalni. Majd azt is megadhatja mennyi időt szeretne ott tölteni, erre 4 lehetősége van 1, 1.5, 2, vagy 3 óra. A név mező automatikusan a felhasználó nevét állítja be, de ez módosítható, nem szükséges, hogy a felhasználó neve megegyezzen a foglalóéval. Ezután a telefonszámot kell megadnia a felhasználónak, aminek formátuma ellenőrizve van, csak szám lehet és a +36 után 9 karakternek kell szerepelnie. A Reserve Boxra kattintás után a jobb oldalon jelennek meg a foglalások. Itt van lehetőség annak törlésére és látható a státusza is, ami vagy Reserved (foglalt), Cancelled (Lemondva), Confirmed (megerősítve). Ezt az asztali alkalmazáson keresztül lehet módosítani.

Select Box

Box 1 4 seats - Reserved	Box 2 4 seats	Box 3 2 seats
Box 4 5 seats	Box 5 4 seats	

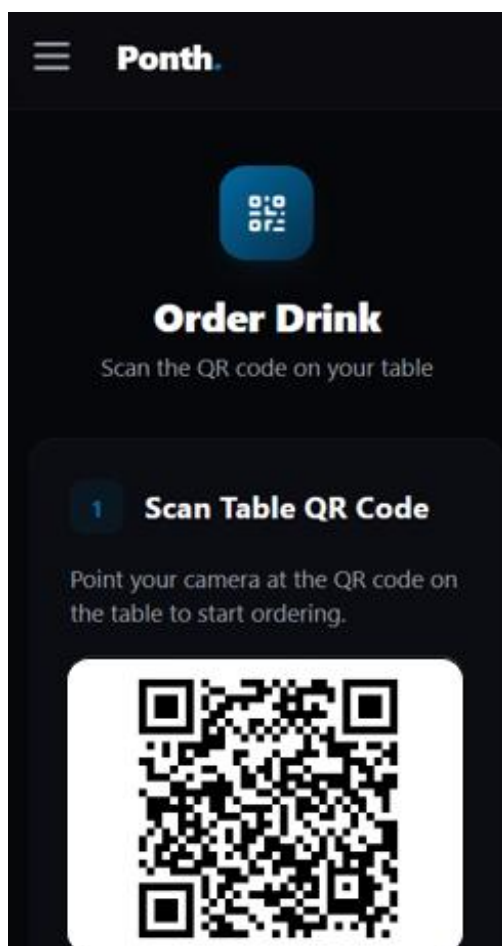
Date

2026. 04. 17. ☐

Time

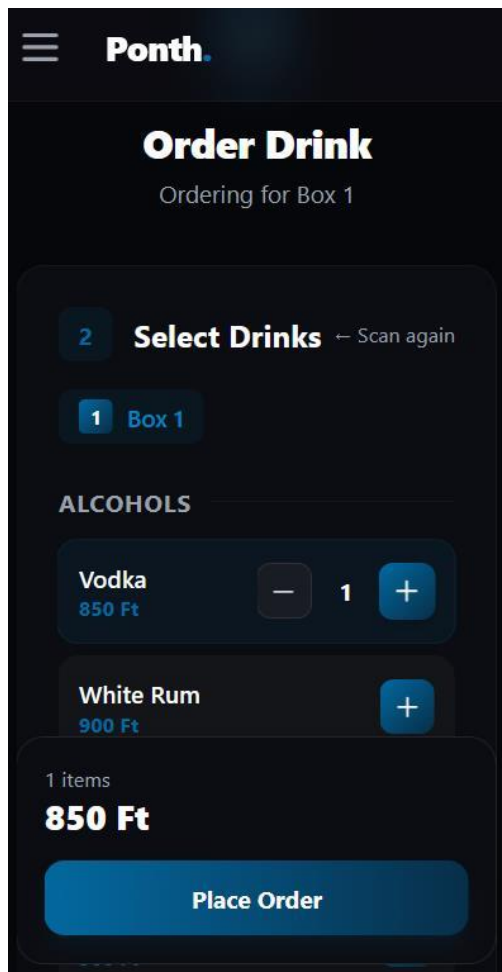
21:00

Ha már foglalt az adott dátumon és időpontban a kiválasztott asztal a szoftver nem engedi elküldeni a foglalást és Reserved-ként (foglalt) jeleníti azt meg.



3.4.6.3. [Rendelés – Order Drink/Order more](#)

Az Order Drink-re vagy Order more-ra kattintva egy olyan felületre visz, ahol az szoftver megnyitja a kamerát és be lehet olvasni az asztalra kihelyezett QR-kódot, ami az asztal számát tartalmazza. Ez a kód elvisz arra a felületre, ahol le lehet adni a rendelést.



Ezen a felületen lehet leadni a rendelést. A + gomb segítségével adható hozzá, a – gombbal pedig törölhetőek a tételek. Az oldalon látható a rendelés aktuális összege. A „Place Order” gomb segítségével pedig véglegesíteni lehet a rendelést.

3.5. Asztali alkalmazás bemutatása

3.5.1. Kezdőoldal

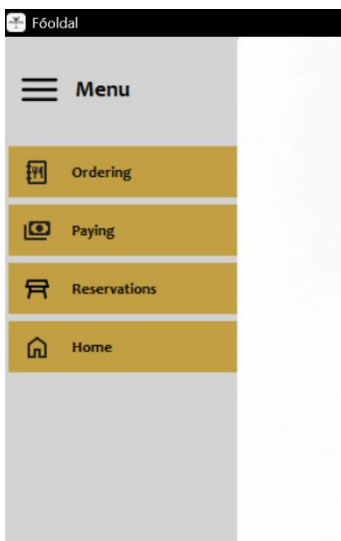


Az alkalmazást elindítva a kezdőoldal nyílik meg, ahol a projekt logója látható. Innen lehet elérni a navigációs menüt és az egyes asztalok rendeléseit. Az oldal alján elhelyezett „English” gomb a későbbi továbbfejlesztés során a nyelv kiválasztására szolgál majd.



Az asztalok rendeléseinek gombjánál megjelenik egy „New” felirat, ezzel jelezve a dolgozók számára, ha valamelyik asztalnál/hoz új rendelést adtak le.

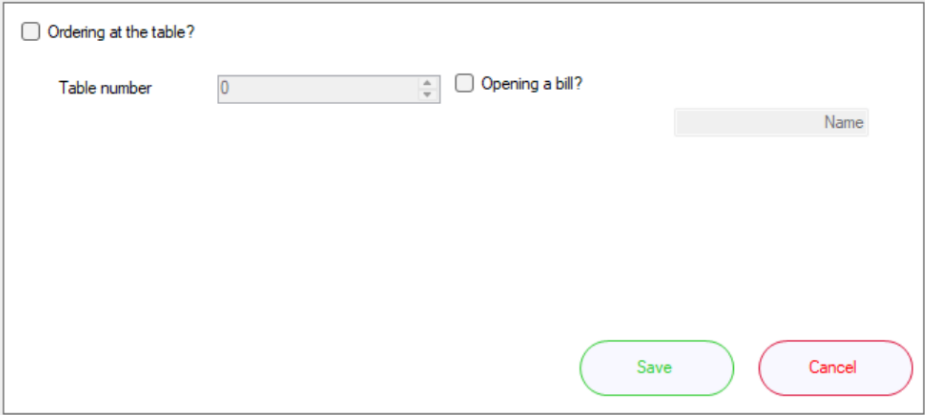
3.5.2. Navigációs sáv



Az bal szélén elhelyezkedő navigációs menüből érhetőek el az alkalmazás különböző funkciói.

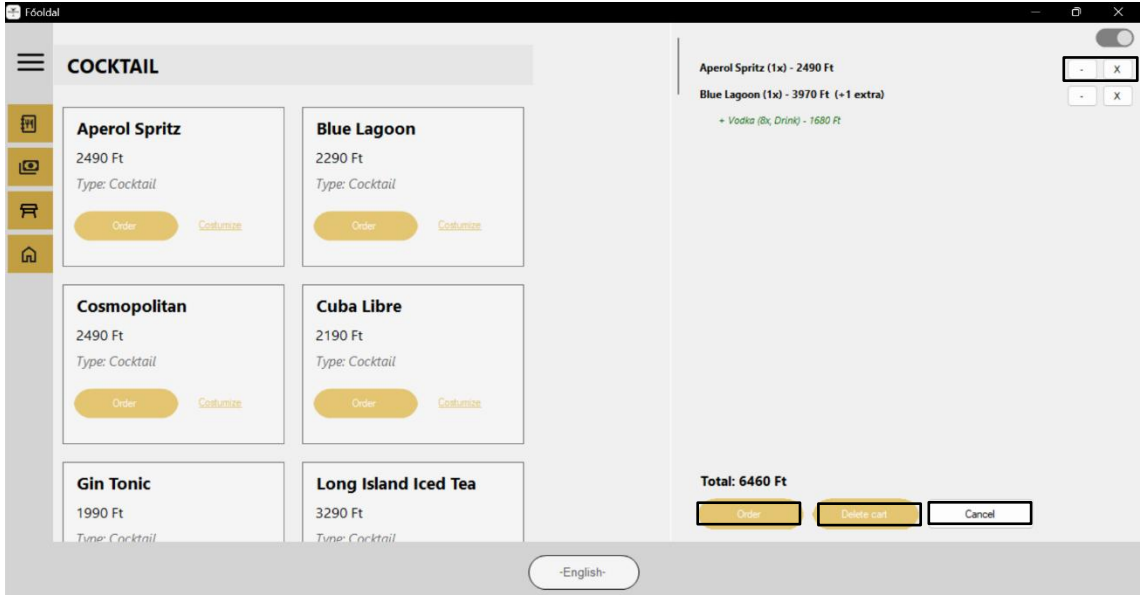
- **Rendelés (Ordering):** A rendelés leadásának felülete
- **Fizetés (Paying):** Fizetési felület
- **Foglalások (Reservations):** Korábbi foglalások
- **Főoldal (Home)**

3.5.3. Rendelés – Ordering



A dialog box for ordering. It contains a checkbox labeled "Ordering at the table?". Below it, there is a "Table number" label and a text input field with the value "0". To the right of the input field is another checkbox labeled "Opening a bill?". Further right is a "Name" label and a text input field. At the bottom right, there are two buttons: "Save" (green outline) and "Cancel" (red outline).

Az „Ordering” menüpontra kattintva először felugrik egy ablak, ahol kiválasztható, melyik asztalhoz szeretne a vendég rendelni, a névre szóló rendelés, mint egy továbbfejlesztési lehetőség kezdetleges megjelenítése.



A screenshot of a web application showing a cocktail menu and an order summary. The menu is titled "COCKTAIL" and lists several drinks with their prices and types. Each drink has an "Order" button and a "Customize" link. The order summary on the right shows the items added to the cart, their prices, and a total. At the bottom, there are buttons for "Order", "View cart", and "Cancel".

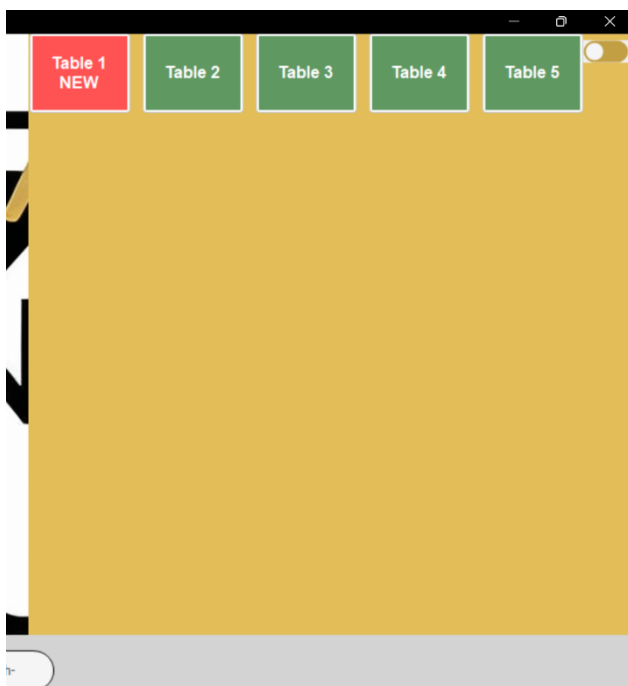
Item	Price (Ft)
Aperol Spritz (1x)	2490
Blue Lagoon (1x)	3970 (+1 extra)
+ Vodka (8x Drink)	1680
Total	6460

Miután a vendég kiválasztotta melyik asztalhoz szeretne rendelni, megjelenik a rendelési felület, ahol látható minden koktél, alkohol és ital. Az italok kártyáján megjelenő „Order” gombbal lehet tételeket hozzáadni a kosárhoz. Lehetőség van a koktélok összetevőinek szerkesztésére is a „customize” felíratra kattintva, aminek módosítása a jobb oldalt elhelyezett „Kosár” ablakban is megjelenik. Az

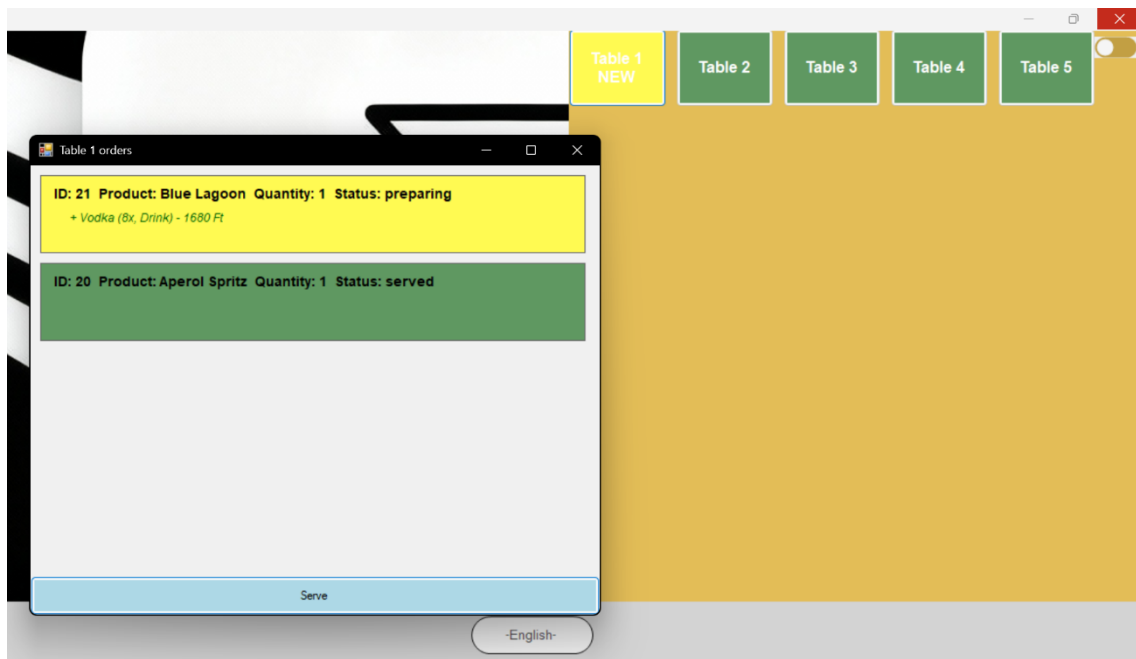
„Order” gombra kattintva van lehetőség a rendelés leadására, a „Delete cart” gombbal pedig az egész kosár tartalmának törlésére. Ha csak egy tételt vagy egy tétel mennyiségét szeretnénk törölni/módosítani a tétel mellett megjelenő +/- gomb segítségével van erre lehetőség. A „Cancel” gombbal pedig kiléphetünk az egész rendelési folyamatból.

The screenshot shows a mobile application interface for a bar or restaurant. It is divided into two main sections: 'CONTAINS' and 'DRINKS'. The 'CONTAINS' section lists four items: 'Aperol' (1200 Ft, 6 cl), 'Prosecco' (1080 Ft, 9 cl), 'Soda Water' (45 Ft, 3 cl), and 'Orange Slice' (80 Ft, 1 piece). Each item has a '+' and '-' button to adjust the quantity. The 'DRINKS' section lists three items: 'CostumeCocktail' (0 Ft, 0 cl), 'Vodka' (0 Ft, 0 cl), and 'White Rum' (0 Ft, 0 cl). At the bottom of the interface, there are two buttons: 'Add to cart' (green) and 'Cancel' (grey).

A „Customize” felírra kattintva érhető el a koktélok módosítása. Itt látható mi az, ami alapból benne van az adott koktélban és azok egységárai és mennyiségei is láthatóak. Növelhetjük vagy éppen csökkenthetjük azok mennyiségét vagy új tételeket is hozzáadhatunk. Az „Add to cart”-ra kattintva pedig hozzáadhatjuk a módosított koktélt a kosárhoz.

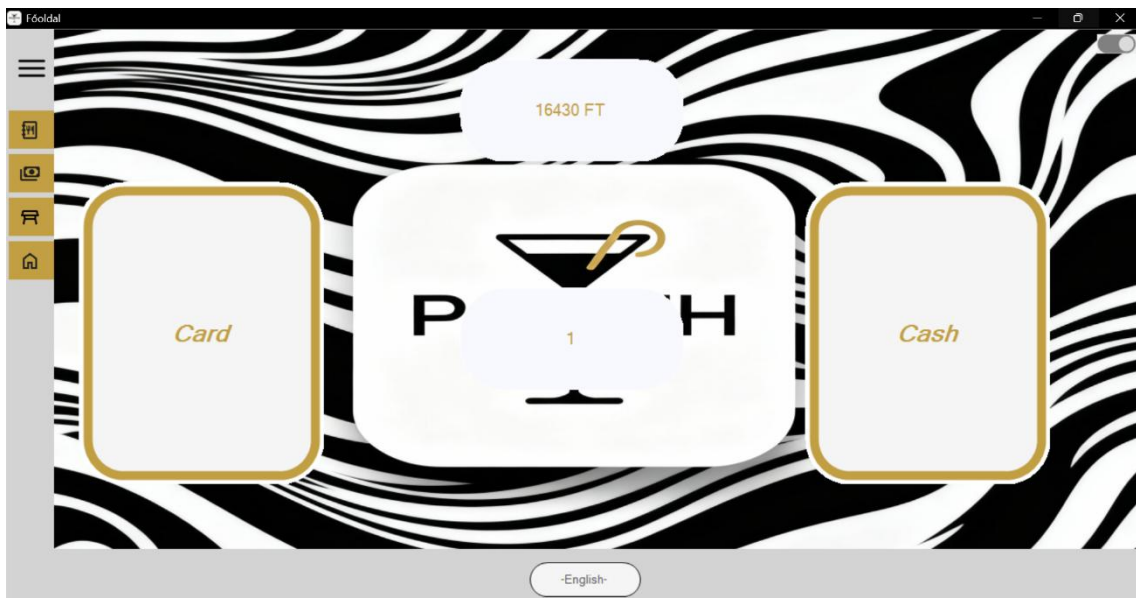


Miután valaki leadta a rendelését, a jobb felső sarokban elhelyezett csúszkánál megjelenik a „New” felirat. Ezt kinyitva látható melyik asztalhoz rendeltek.



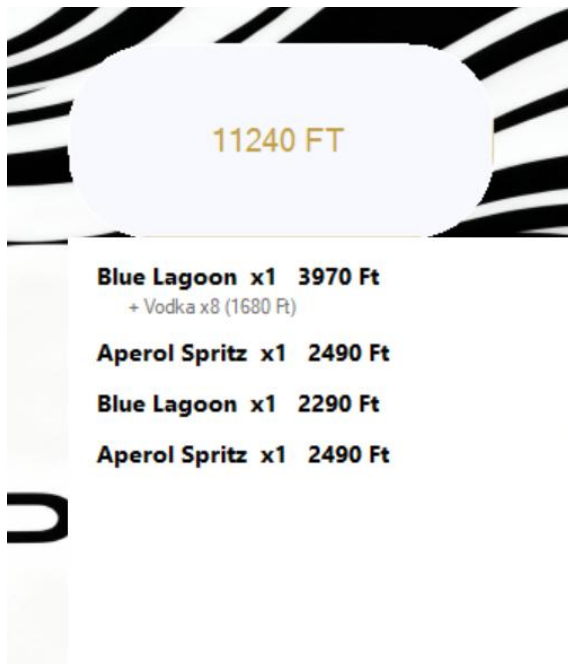
Miután rányomtunk az adott asztalra, láthatóak a tételek, amiket rendeltek. Ha rányomunk egy tételre annak státusza „Preparing”-ra (készül) változik, még egy kattintásra pedig „Served”-re, azaz készre. Az alul elhelyezkedő „Serve” gomb segítségével pedig minden tétel státuszát készre állíthatunk.

3.5.4. Fizetés – Paying



A „Paying” menügombra kattintva az alkalmazás elnavigál minket a fizető felületre, a középen elhelyezett gombbal lehet kiválasztani, melyik asztalt szeretnénk fizettetni, felül megjelenik a rendelés teljes összege, amire rákattintva látható a részletes rendelés. Ki lehet választani, hogy kártyával (Card) vagy

készpénzzel (Cash) szeretne fizetni a vendég. Miután kiválasztottuk az alkalmazás visszalép a főoldalra.



Itt láthatóak a fizetésnél, az összeg gombjára kattintva a rendelés részletei pontos mennyiségekkel és árakkal.

3.5.5. Foglalások – Reservations

A screenshot of a web application interface titled 'Főoldal'. It features a search bar at the top with a 'Search' button. Below the search bar is a table with reservation data. The table has columns for Name, Phone number, Date, Time, Duration time, Table, and Status. A single reservation is listed for 'Bek' with phone number '+36324234232', date '2026. 04. 30.', time '21.00.00', duration '60', table '1', and status 'Reserved'. A sidebar on the left contains icons for menu, camera, list, and home.

Name	Phone number	Date	Time	Duration time	Table	Status
Bek	+36324234232	2026. 04. 30.	21.00.00	60	1	Reserved

A „Reservations” aloldalon jelennek meg a webes felületen regisztrált foglalások. Látható a foglaló neve, telefonszáma, a foglalt időpont, az ott eltölteni kívánt idő, az asztalszám és a foglalás státusza. A lap tetején helyezkedik el egy kereső sáv, ahol név alapján lehet szűrni a foglalásokat. A program a gépelés közben már automatikusan szűri a keresett neveket.

4. Továbbfejlesztési lehetőségek

A rendszer jelenlegi funkcionalitása stabil alapot biztosít a bistró napi működéséhez, azonban a felhasználói élmény fokozása és az üzleti hatékonyság növelése érdekében az alábbi fejlesztési irányok tűzhetőek ki célul:

- **Multilokális és többnyelvű támogatás:** A szoftver nemzetközi környezetben vagy turistaforgalomban való alkalmazhatóságát segítené a többnyelvű felület bevezetése. Ez nem csupán a kezelőfelület lefordítását, hanem az itallap és az összetevők dinamikus nyelvi váltását is magában foglalná.
- **Személyre szabott kiszolgálás (Névre szóló rendelés):** A felhasználói profilok kiterjesztésével lehetőség nyílna arra, hogy a rendelések ne csak asztalszámhoz, hanem konkrét nevekhez kötődjenek. Ez személyesebbé tenné a vendéglátást, és segítené a pincérek munkáját a nagyobb társaságok kiszolgálásakor.
- **Közösségi receptmegosztás (Social Space):** Egy olyan virtuális közösségi tér kialakítása, ahol a regisztrált felhasználók közzé tehetik saját kreációikat. A vendégek böngészhetnék, értékelhetnék vagy akár „lájkolhatnák” egymás egyedi koktéltreceptjeit, közösségi élménnyé emelve az italkészítést.
- **„Közönség kedvenc” itallap-frissítés (Kedvencek az alapmenübe):** A rendszer statisztikai adatai és a közösségi visszajelzések alapján a legnépszerűbb egyedi koktélok automatikusan (vagy adminisztrátori jóváhagyással) bekerülhetnének az állandó kínálatba. Így a vendégek közvetlen hatást gyakorolhatnának a bistró aktuális választékára.
- **Dinamikus menümenedzsment:** Az adminisztrátori felület továbbfejlesztése, amely lehetővé tenné a szezonális kínálat gyors és rugalmas kezelését. Ide tartozna az árak időszakos módosítása (pl. Happy Hour beállítása), az ideiglenes készlethiányok azonnali jelzése a vendégoldalon, új italok hozzáadása vagy meglévők módosítása, valamint a vizuális étlap-elrendezés testreszabása.

5. Irodalomjegyzék

5.1. Könyvek, tankönyvek

1. Szerző: Matt Stauffer; cím: Laravel: Up & Running: A Framework for Building Modern PHP Applications (3rd Edition); kiadó: O'Reilly Media; kiadás éve: 2023.; ISBN: 978-1098138950
2. Szerző: Josh Lockhart; cím: Modern PHP: New Features and Good Practices; kiadó: O'Reilly Media; kiadás éve: 2015.; ISBN: 978-1491905012
3. Szerző: Szeifert Ferenc; cím: Webfejlesztés PHP-vel; kiadó: BBS-Info; kiadás éve: 2023.; ISBN: 9786156347183
4. Szerző: Bill Karwin; cím: SQL Antipatterns: Avoiding the Pitfalls of Database Programming; kiadó: Pragmatic Bookshelf; kiadás éve: 2010.; ISBN: 978-1934356555
5. Szerző: Martin Kleppmann; cím: Designing Data-Intensive Applications; kiadó: O'Reilly Media; kiadás éve: 2017.; ISBN: 978-1449373320
6. Szerző: Joe Celko; cím: SQL for Smarties: Advanced SQL Programming; kiadó: Morgan Kaufmann; kiadás éve: 2014.; ISBN: 978-0128007617
7. Szerző: Robert C. Martin; cím: Tiszta kód – Az okos programozás kézikönyve; kiadó: Panem Könyvkiadó; kiadás éve: 2010.; ISBN: 9789635455300; fordító: Sághy Erna
8. Szerző: Robert C. Martin; cím: Tiszta architektúra – A szoftverfejlesztés és -tervezés kézikönyve; kiadó: Jaffa Kiadó; kiadás éve: 2019.; ISBN: 9789634751410; fordító: Bakonyi Zoltán
9. Szerző: Robert C. Martin; cím: Tiszta tesztelés – Útmutató a hatékony szoftvertesztekhez; kiadó: Jaffa Kiadó; kiadás éve: 2023.; ISBN: 9789634757139; fordító: Bakonyi Zoltán
10. Szerző: Steve McConnell; cím: Code Complete: A Practical Handbook of Software Construction (2nd Edition); kiadó: Microsoft Press; kiadás éve: 2004.; ISBN: 978-0735619678
11. Szerző: Malcolm McDonald; cím: Web Security for Developers: Real Threats, Practical Defense; kiadó: No Starch Press; kiadás éve: 2020.; ISBN: 978-1718500440

12. Szerző: Dino Esposito; cím: Clean Architecture with .NET; kiadó: Microsoft Press; kiadás éve: 2024.; ISBN: 9780138250263
13. Szerző: Mark J. Price; cím: C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals (8th Edition); kiadó: Packt Publishing; kiadás éve: 2023.; ISBN: 978-1837635870
14. Szerző: Angster Erzsébet; cím: Objektumorientált tervezés és programozás (Java és C# példákkal); kiadó: 4KÖR; kiadás éve: 2013.; ISBN: 9789630855211
15. Szerző: Aditya Bhargava; cím: Algoritmusok okosan – Illusztrált útmutató programozóknak; kiadó: HVG Könyvek; kiadás éve: 2021.; ISBN: 9789633049969; fordító: Laki Beáta
16. Szerző: Eric Ries; cím: The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses; kiadó: Crown Business; kiadás éve: 2011.; ISBN: 978-0307887894
17. Szerző: Steve Krug; cím: Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability (3rd Edition); kiadó: New Riders; kiadás éve: 2014.; ISBN: 978-0321965516

5.2. Tutoriálok, online források

1. Laravel Daily (Povilas Korop) – [How to Structure Your Laravel Project: 5 Real Examples](#)
2. Laravel - [Laravel](#)
3. FreeCodeCamp.org - [Relational Database Design: Full Course](#)
4. FullStackOpen - [Deep Dive Into Modern Web Development](#)
5. GeekForGeeks - [DBMS Tutorial](#)
6. W3Schools - [C# Tutorial](#)
7. Siminium - [Modern Dashboard UI in C# WinForms | Guna2 UI Project Tutorial](#)

6. Mellékletek

6.1. Publikus Github repository

Github Repository link: <https://github.com/NemethRajmondAdam/ponth.git>

A teszteléshez a felhasználók:

- **Felhasználó:** username: peldabela@gmail.com, password: 123456
- **Admin:** username: test@gmail.com, password: 123